ENCORE: Enabling the Wide Adoption of Oblivious Computation

Overview

The proposed research aims to accelerate the wide-scale adoption of oblivious computation. Unlike most existing confidential computing offerings, oblivious computation is distinct in that it protects not only the data contents, but also the access patterns to data, which often directly reflect the users' intentions in high-value applications such as blockchains and Large-Language Models (LLMs). To achieve this, the proposed technology combines Trusted Execution Environment (TEEs) and Oblivious RAM (ORAM).

Keywords: Oblivious RAM, Cryptography, Trusted Execution Environment, Trusted Computing

Intellectual Merit

This Small Business Innovation Research Phase II project has the following intellectual merits.

- *Oblivious accesses to public databases.* The proposed work aims to develop and launch an "oblivious accesses to public databases" service. By delivering strong privacy protection while maintaining competitive performance and pricing as existing non-private services, the resulting service is expected to successfully convert users of non-private services today and generate revenue.
- *Oblivious container.* The proposed work also aims to develop a high-assurance containerization technology enhanced with oblivious storage, and make it available to cloud customers through the marketplaces of major cloud platforms such as Microsoft Azure or Google Cloud. This allows enterprises and individuals to outsource data and computation to the cloud without risking the exposure of sensitive information to co-resident cloud tenants or the cloud platform itself.
- *Oblivious STL: oblivious algorithm meets hardware enclaves.* Another objective is to develop highly optimized oblivious implementations of common data structures and utility algorithms, and make them available to the community through open source. The intellectual merit of this task is enclave-friendly adaptations of existing oblivious algorithms. Most existing oblivious algorithms in the literature focus on optimizing the *computational overhead*. However, in TEE environments, the *page swap* overhead is also a major bottleneck. The proposed Oblivious STL effort will design a new class of TEE-friendly oblivious algorithms that improve the page swap overhead asymptotically, leading to one or more orders of magnitude concrete performance improvement.
- *Compiler-algorithms co-design.* The proposed work will marry oblivious algorithms and formal methods in innovative ways to achieve 1) aggressive compile-time optimizations leading to automated synthesis of highly efficient oblivious implementations; and 2) scalable formal verification of a program's obliviouness at compile time.

Broader Impacts

The proposing team is working with industry leaders such as Signal and Ethereum to promote the adoption of oblivious computation as a fundamental infrastructure-level service in cloud computing and blockchains. Support from these major stakeholders will help establish oblivious computation as a new industry paradigm and advance a future where privacy is "default-on." Stronger privacy protection can in turn foster greater willingness to share data, thus unlocking new functionalities and utilities through AI and data mining.

The open-source libraries developed through this project will serve as a public good, making oblivious computation accessible to the broader community, including non-expert developers. The proposing team will lead an open-source consortium on oblivious computation and will invite key stakeholders — including Signal, Ethereum, Meta, and others — to collaboratively contribute to the maintenance of these libraries.

1 Results of the Phase I Project

1.1 Background on Oblivious Computation

We first persent some background on our oblivious computation technology called ENCORE (short for "ENCrypted and Oblivious computation made a REality"), to put our Phase I accomplishments in context.

ENCORE, developed by our company Oblivious Labs, is an *oblivious computation* framework based on Trusted Execution Environment (TEE) and Oblivious RAM (ORAM) [GO96, Gol87, SvDS+13, WCS15]. Unlike many standard encrypted database solutions [PRZB12, mon, tde] which protect only the *database contents*, ENCORE protects both the *database contents* as well as *queries to the database*. Specifically, ENCORE relies on ORAM [GO96, Ost90, SvDS+13, WCS15] to provide provably oblivious accesses to the data, such that users can query the database efficiently without disclosing the access patterns.

In many real-life applications, the access patterns are highly sensitive and directly leak the users' intentions. For example, in private contact discovery, the database locations accessed directly reveal the user's contacts even when the database contents are encrypted. For exactly this reason, Signal [sigb] has implemented the Path ORAM algorithm [SvDS+13] (co-invented by our Chief Scientist Shi in her earlier academic research). A similar concern arises with Large Language Models (LLMs). A widely used technique known as Retrieval Augmented Generation (RAG) reformulates a user's sensitive query to retrieve relevant information from a large knowledge base. In this case, the resulting access patterns also directly reveal the user's intentions.

1.2 Phase I Accomplishments

Open source releases. During Phase 1, we released the following open source libraries and toolchains.

- **Oblivious key-value store.** We released a highly optimized oblivious key-value store implementation [Labb]. We started with providing a key-value store abstraction because it is desired in many applications of oblivious computation, including those needed by our existing and potential customers Signal, Meta, Ethereum, Flashbots, and Worldcoin.
- **Oblivious STL.** Oblivious RAM (ORAM) [GO96, Gol87, SCSL11, SvDS+13, WCS15] is a generic approach for compiling *any* program into an oblivious counterpart. Although ORAM is sufficient for enabling general-purpose oblivious computation, customized oblivious algorithms tailored for specific classes of computation tasks often outperform generic ORAM asymptotically and/or concretely. This motivates our Oblivious STL effort, which aims to provide a fast, oblivious counterpart of a standard library similar to C++'s STL and Rust's `std`. During Phase I, we have successfully kicked off our oblivious STL project, and our code has been released through open source [ost]. So far, we have developed fast, oblivious implementations of standard data structures such as ordered map, unordered map, array, queue, stack, priority queue, as well as common utility algorithms such as sorting, shuffling, filtering, and union find.
- **Reproducible benchmarks for oblivious computation.** We released an open-source repository containing reproducible benchmarks of various ORAM and oblivious algorithmic building blocks [Labd], including the implementations by our company, and those of Signal, Meta, Mobilecoin, and others.
- **Oblivious EVM opcodes.** EVM is the virtual machine standard for Ethereum's smart contracts. We released an oblivious implementation of EVM's 149+ opcodes [Laba]. More specifically, we modified the underlying implementation that interprets and executes every EVM opcode, and ensured that 1) all memory accesses go through ORAM; and 2) if there exists any conditional dependent on secret data, then all branches exhibit identical memory and instruction traces. This lays the groundwork for enabling

Table 1: **Average query latency of our oblivious key-value store and other implementations.** The running times are in microseconds (μs). We use 56-byte keys and 8-byte values like in Signal's scenario. Note that 2^{28} to 2^{30} entries are sufficiently large to represent Signal-scale and Ethereum-scale scenarios. All experiments were run on a single Google Cloud C3 VM with 16 cores and 176GB of memory.

data size	Oblix	Enigmap	Signal	Meta	**Oblivious labs**
2^{14}	> 2000	> 100	14	116	3.1
2^{20}	> 3000	> 250	38.7	572	6.8
2^{26}	> 8000	> 500	91	922	12.2
2^{28}	-	> 600	121	1510	14.2
2^{30}	-	> 800	139	-	17.1

an end-to-end "private order flow" architecture for Ethereum — see our "Commercialization Plan" for a more detailed explanation of the use case.

- **Demo: oblivious accesses to blockchain.** To showcase fast oblivious accesses to blockchain data, we have created a demo that is an oblivious ERC20 token balance checker [Labe]. ERC20 tokens are derived cryptocurrencies implemented on top of Ethereum. Our demo lets a user check its balance without identifying the queried account. The code for our demo has been released through open source [Labc].

Performance evaluation. During Phase I, we significantly improved the performance of our oblivious key-value store. To the best of our knowledge, ours is the fastest oblivious key-value store implementation available today. We evaluated our implementation and compared its performance with several other implementations, including 1) a production implementation created by Signal [siga]; 2) an implementation by Meta [met] intended for production use; 3) our own prior research prototype Enigmap [TGS23], developed in our basic research at CMU; and 4) Oblix [MPC$^+$18], a state-of-the-art research implementation developed by others.

As shown in Table 1 and Table 2, even on a single machine, we can achieve **14.2** to **16.2**μs per key-value lookup on databases of Ethereum's or Signal's scale, and **323K** queries/sec throughput under stress load. In comparison with Signal or Meta's implementations, our oblivious key-value store achieves more than **8**$\times$ and **106**$\times$ improvement in query latency and **5.4**$\times$ and **536**$\times$ improvement in throughput, respectively[1].

In comparison with our own prior research prototype Enigmap [TGS23] — which represents our status quo when writing the Phase I proposal — our latest production implementation achieves **39 - 180**$\times$ improvement in throughput and **269**$\times$ improvement in query latency on Ethereum- and Signal-scale databases.

These significant performance improvements come from several major algorithmic ideas that we developed since the writing of the Phase I proposal: 1) using an oblivious cuckoo hash table rather than an oblivious map to realize an oblivious key-value store; 2) replacing Path ORAM [SvDS$^+$13] with a heavily optimized Circuit ORAM [WCS15] implementation, to reduce the computational overhead; and 3) a multi-threaded parallel implementation that allows us to scale up the throughput with the number of threads.

Besides improving the online query costs, we also developed a faster ORAM initialization algorithm based on algorithmic ideas initially proposed in our earlier work Enigmap [TGS23]. Our ORAM initialization algorithm outperforms that of Signal's by roughly **5**$\times$.

Quantifiable metrics of feasibility in real-life applications. Through our Phase I development and performance benchmarking efforts, we obtained quantifiable measures of feasibility in real-life scenarios:

[1]It is worth noting that Signal's current implementation contains minor security flaws—specifically, it inadvertently leaks information by exposing the exact stash size. In private communication with their technical lead, we confirmed their intention to address this issue. Once corrected, their system's performance is expected to degrade.

Table 2: **Throughput under stress load of our oblivious key-value store and other implementations.** We use 56-byte keys and 8-byte values, and the experimental machine is the same as in Table 1.

data size	Oblix	Enigmap	Signal	Meta	**Oblivious Labs**
2^{14}	< 500	< 10000	594139	8573	1026174
2^{20}	< 250	< 4000	279803	1748	693988
2^{26}	< 125	< 2000	93631	1084	376622
2^{28}	-	< 1600	72593	662	355170
2^{30}	-	< 1200	59723	-	322865

- *Scenario 1: protecting all Uniswap transactions in private block building.* If we want to use ORAM during block building to secure the accesses of all Uniswap transactions, how much delay are we looking at? This is a typical scenario that our partner Flashbots cares about. On average, each Ethereum block has about 20 Uniswap transactions. Each Uniswap transaction makes 3 memory accesses (`sload`/`sstore` operations) that are sensitive. Using our microbenchmarks above, the projected delay for building a block would be 0.97ms, assuming that we store the entire Ethereum state in ORAM. The latency can be further reduced to 0.5ms by storing each Uniswap contract in a separate ORAM, at the price of leaking which pair of tokens is being traded.
- *Scenario 2: Signal's scenario.* Signal's production deployment of Path ORAM [siga] has already demonstrated the feasibility of using ORAM on databases with hundreds of millions of users. Prior to adopting Path ORAM, Signal relied on a naïve batched linear scan approach for privacy, which required approximately 500 servers. By switching to Path ORAM, they reduced this requirement to just 6 servers, resulting in a 100× reduction in computing costs. Our implementation further improves upon this, achieving approximately $8\times$ lower query latency compared to Signal's implementation, clearly demonstrating its scalability to large-scale datasets.
- *Scenario 3: Ethereum's scenario.* Ethereum is our newly acquired partner during Phase I, and they would like us to build oblivious access capabilities into Ethereum's full nodes. Ethereum's data scale is similar to Signal's, with slightly different key/value sizes. So the aforementioned performance numbers at 2^{30} entries are also indicative of the performance for Ethereum's use case.

In summary, our Phase I R&D firmly established the practicality of our approach on large-scale databases, even for performance-hungry application scenarios such as block building, which is sensitive to even millisecond-level delays.

Progress on commercialization. During Phase 1, we solidified existing partnerships and performed new customer discovery. We interviewed 40 companies including including 10 in cloud computing, 15 in blockchain and decentralized finance, 8 in AI, and 7 in healthcare and traditional finance. Below are some selected use cases of our existing or new partners whom we are finalizing contracts with:

- *Signal.* Signal's use case, namely, private contact discovery, was already mentioned earlier as a motivating example. Because our implementation significantly outperforms Signal's in both latency and throughput, Signal is interested in migrating to our code base. We have been working closely with their R&D team. As a first step towards this migration, they already provided us with a synthetic user database which matches the size and dimensions of their actual dataset, but filled with garbage data. We are now conducting extensive performance benchmarking to prepare for the migration.
- *Ethereum.* Ethereum wants to enable oblivious accesses to their blockchain. This can in turn lead to many applications such as private order flow — see our "Commercialization Plan" for more details. We

have had several conversations with key stake-holders including Ethereum's founder Vitalik Buterin. Impressed with the high performance delivered by our solution, Ethereum will contract us to integrate oblivious Remote Procedural Calls (RPC) into Ethereum's full nodes.

- *Flashbots.* Flashbots has been our key partner since the formation of our company. We have successfully executed our first contract with Flashbots, where we developed demos and conducted a feasibility test for using ENCORE in their intended private order flow application. We showcased our results to them in several talks hosted by Flashbots. These results have also been published in our blog post joint with Flashbots [TGSM25]. Flashbots's key stakeholders have expressed interest in continuing their support of our company.
- *Worldcoin.* Worldcoin is a potential customer we newly discovered during Phase I. Our ENCORE technology is a perfect fit for their application and we are now negotiating a contract with them, to help them roll out our technology. Specifically, Worldcoin wants to provide a universal cryptographic identity (called World ID) for users. Currently, users' identities are directly exposed through access patterns during the authentication process, and ENCORE provides the perfect and irreplaceable solution to their problem — see our "Commercialization Plan" for more details.
- *0xPARC.* 0xPARC is also one of our key partners since the formation of our company. 0xPARC's customers include major players in the gaming industry (e.g., EVE online) who are interested in using cryptography to decentralize their games, and oblivious computation is a key technology in this roadmap. During Phase I, we renewed our contract and collaboration with 0xPARC.

Letters of support. The success of our Phase I commercialization efforts can be evidenced by the attached letters of support from Ethereum, Signal, Worldcoin, Flashbots, and 0xPARC, showing their commitment to partner with us and become a contributor to our oblivious computation open-source consortium.

2 Technical Objectives, Approach and Work Plan

Overall R&D roadmap. We first present our overall R&D and product roadmap, which helps to contextualize our proposed Phase II activities as well as our commercialization plan to be presented later. To accelerate the wide-scale deployment of oblivious computation, our roadmap includes the research and development of the following products or technologies, along with their intended timeline:

1. **Oblivious key-value store** (Phases I and II): In Phase I, we focused on developing a high-performance oblivious key-value store. In Phase II, we will expand our oblivious key-value store into a full "oblivious accesses to public databases" service, and bring it to market. We will first target blockchain data, and provide a Remote Procedure Call (RPC) service for obliviously accessing on-chain information.
2. **PODS oblivious containers** (Phase II and beyond): Containerization technology [FMN20, dev, SAD19, DPX+21] lets users run legacy operating system (OS) and application code in an isolated, high-trust environment. Containers provide a convenient method for users to effortlessly migrate their existing applications to cloud platforms such as Amazon AWS, Microsoft Azure, and Google Cloud [aws, azub, gooa]. Unfortunately, existing containerization technology fails to deliver the strong isolation guarantees it promises. Applications running in today's containers directly expose to the host OS, the hypervisor, or the networked storage provider their access patterns to data.

   In Phase II, we plan to develop a new containerization technology called PODS (short for "Privatizing containers with Oblivious Data Storage"), which provides significantly stronger isolation guarantees

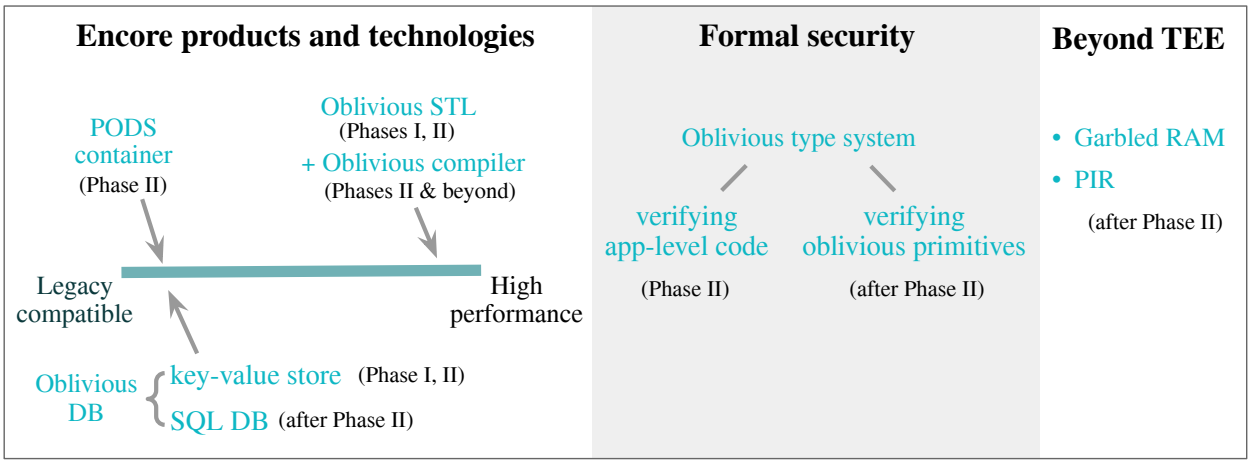


Figure 1: **Our overall R&D roadmap**: including past accomplishments, Phase II statement of work, and long-term objectives.

than traditional containers by plugging access pattern leakages. PODS inherits the most compelling advantage of traditional containers, that is, full *legacy compatibility*. This way, non-private legacy services can effortlessly migrate to PODS and start offering privacy protection.

3. **Oblivious SQL database** (after Phase II): Besides the key-value abstraction, it will also be desirable expand our oblivious database offering and support richer abstractions. Since SQL databases are among the most widely adopted databases, we plan to develop an oblivious SQL database. We currently plan to defer this effort to after Phase II since oblivious key-value store is a better fit for our existing or prospective customers's needs.

4. **Oblivious STL** (Phases I and II): As mentioned in Section 1, in Phase I, we have launched our Oblivious STL open-source library [ost], which offers highly optimized oblivious implementations of a suite of commonly used data structures and algorithmic building blocks. In Phase II, we plan to continue this effort with two main goals: 1) further optimize the existing algorithmic building blocks, and implement multi-threaded, parallelized versions of the existing algorithms; and 2) expand the current Oblivious STL to support richer algorithmic building blocks.

5. **Oblivious compiler** (Phase II and beyond): As our customer base grows — particularly with clients who seek our support to customize and accelerate their oblivious implementations — we want to make such efforts more scalable in terms of human expertise. To this end, we will build an oblivious compiler that takes source programs written in a high-level source language, and emits highly optimized oblivious implementations. With such an oblivious compiler, our experts only need to augment our customers' programs with lightweight annotations which guide the compiler in making optimizations — these annotations are not required for our compiler to work, but they can provide hints that lead to faster target implementations. In Phase II, we plan to develop an oblivious compiler for Rust along with accompanying annotations. After Phase II, we also plan to provide support for other popular source languages beyond Rust.

6. **Formally verified obliviousness through typing** (Phase II and beyond): To achieve formally verified security, in Phase II, we will enhance Rust or a Rust-like language with a memory-trace oblivious type system [LHS13, LWN+15, LHH+15].

7. **Replacing TEE with cryptography** (after Phase II): While there is significant momentum to adopt TEEs in both AI and blockchain ecosystems — driven in part by their engineering simplicity and high

performance — several of our partners, including Signal, Ethereum, and Flashbots, have made it clear that removing reliance on TEEs remains a longer-term goal. Our vision aligns with this trajectory. In the long run, we aim to eliminate the TEE from the trusted computing base by combining oblivious computation with a "cryptography-enabled secure computer" built from multi-party computation (MPC). See Section 2.5 for more details.

2.1 Oblivious Accesses to Public Data

Building on our oblivious key-value store developed in Phase I [Labb], we will develop a full-fledged service featuring "oblivious accesses to public databases". To start with, we will target blockchain data, and build an oblivious counterpart of existing RPC services such as Infura [infc] and Alchemy [alca]. Such RPC services form a fundamental part of the blockchain's infrastructure, and have a well-established market. They are widely used by applications such as online wallets, analytics platforms, indexers, and smart contract monitoring tools.

Unfortunately, existing RPC services provide no privacy protection. Our solution offers a compelling alternative by delivering strong privacy guarantees without compromising on performance or pricing. This positions us to rapidly attract and convert developers and users currently relying on non-private services. Specifically, such an RPC service provides APIs that allow the caller to make various types of queries, e.g., get the balance of an address at a specific block, get the details of a transaction by its hash, etc. We will augment existing services with the following security guarantees: 1) *obliviousness*, that is, the callers' queries are not exposed, thus hiding the users' identities, trade intentions, and transactional counterparties; and 2) *verifiability*, that is, we can optionally return a cryptographic proof (e.g., a Merkle proof) that vouches for the correctness of the result.

The anticipated deliverable for this task is a live, privacy-preserving RPC service supporting a rich set of API calls for querying on-chain information, along with the code for a single-machine implementation publicly released through open source. The task can be further broken down into the following milestones.

Milestone 1.a: baseline implementation. During the first six months, we will create a baseline implementation that treats the blockchain's state as a big key-value store, so that we can directly build on top of the oblivious key-value store [Labb] developed in Phase I. To provide verifiability guarantees, the value field of each address will be augmented with a corresponding Merkle proof. We expect the baseline version to support common Ethereum's JSN-RPC queries such as `eth_getBalance`, `eth_getStorageAt`, and `eth_getProof` (EIP-1186 [eip]).

Milestone 1.b: algorithmic improvement. In our baseline implementation, flattening the Merkle proofs into the key-value entries incurs a logarithmic overhead in space, since the Merkle proofs of different addresses share common components, which end up being stored multiple times. Our next step is to design and implement algorithmic optimizations to eliminate this logarithmic space blowup. Specifically, Ethereum employs a Merkle Patricia Tree (MPT) to enable authenticated data retrieval. In our prior work on oblivious data structures [WNL+14], we introduced a technique that aligns the recursion structure of ORAM with the underlying logical data structure. We now plan to adapt this approach to Ethereum's MPT to achieve a logarithmic reduction in space overhead. We expect this milestone to be completed by the end of Year 1.

Milestone 1.c: expand APIs supported. We plan to launch our oblivious RPC service by the end of Year 1, initially supporting a core set of common queries. In Year 2, we will focus on expanding the range of supported APIs to evolve the service into a full-featured offering, focusing on providing oblivious versions of all the Ethereum's JSON-RPC state and history accessing methods.

2.2 PODS Oblivious Container

Cloud computing provides economies of scale, enabling businesses and individuals to outsource data storage and computation while paying only for the resources they consume. In particular, containerization technology [doc, lxc, lxd, gvi, ATG+16, gsc] offers a convenient abstraction for migrating unmodified legacy services and computation to the cloud. However, as cloud adoption grows, so do concerns over the privacy of data and computation outsourced to the cloud. In response, industry leaders such as Google, Apple, and Microsoft have raced to announce imminent plans to build the next-generation privacy-preserving cloud platforms for AI [app, azua, OR]. Such private AI platforms leverage Trusted Execution Environments (TEEs) safeguard tenants' sensitive information — not only from co-resident tenants but also from the cloud providers themselves.

Unfortunately, existing containerization technology offered by mainstream cloud services [gooa, aws, nit, azub] fail to provide strong isolation guarantees. Alarmingly, the trusted domain's accesses to local or networked disks are directly exposed to the underlying OS, hypervisor, or the networked storage provider.

We propose extending containerization technology with an oblivious storage backend, thus preventing the leakage of secret information through access patterns to any untrusted component, including the host OS, hypervisor, co-resident containers, as well as the cloud hosting service itself. In this proposal, we use the name POD (plural form: PODS) to refer to a container "Privatized with Oblivious Data Storage". In addition to providing oblivious data access, we will ensure storage integrity, preventing untrusted domains from tampering with mission-critical data belonging to a trusted domain.

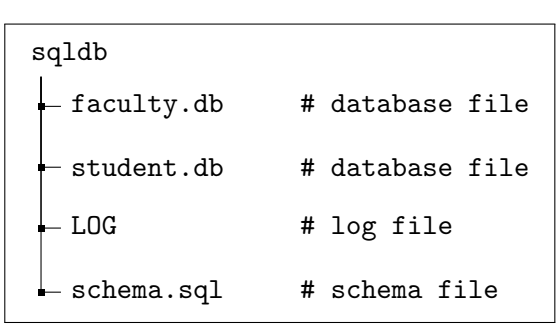
2.2.1 Motivating Example

Imagine that a company henceforth called Podsy (e.g., offering instant messaging or social networking) wants to outsource its data and computation to the cloud. To make the example more concrete, suppose that Podsy stores its data in some database, such as a SQL database (see Figure 2a) or a key-value store (see Figure 2b). With our container coffering, Podsy can simply spin up a PODS container on the cloud provider, and easily migrate their existing applications as container images. As shown below, the only configuration necessary is to create a manifest file that declares appropriate security annotations on some relevant files and directories, as described below:

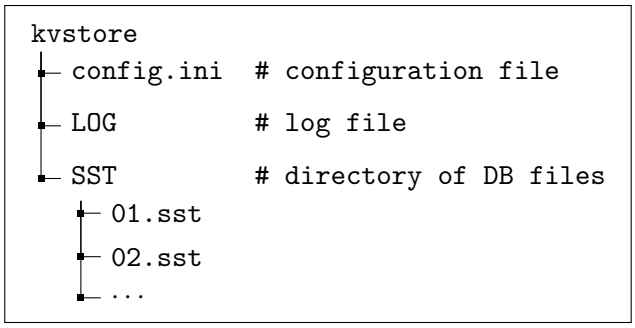
Security annotations on files. Podsy can annotate the files with the following types:

1. `Cleartext:` The file need not be secured and can be stored in cleartext. For example, in Figure 2a, the schema file `schema.sql` was only used to create the databases' schemas (which is not private), and accesses to it are always sequential. In this case, the user can specify its type as `cleartext`.
2. `Encrypted:` The file's contents are encrypted, but accesses to the file need not be protected. For example, in Figure 2a, the log file is always accessed sequentially, but its contents should be protected. In this case, the user can specify its type as `encrypted`.
3. `Oblivious:` The file's contents are encrypted and accesses to the file are protected. For example, in Figure 2a, the database files store sensitive information and accesses to the files are also sensitive. In this case, the user can specify its type as `oblivious`.

Security annotations on directories. Similarly, we allow security annotations at the directory level. We require that *the files within a directory have security labels that are at least as strong as the directory's label*. For example, if a directory is labeled as `encrypted`, the files within it may be labeled as `encrypted` or `oblivious`, but not `cleartext`. Figure 2b shows a key-value store where the key-value pairs are stored in a set of `*.sst` files under the `SST` directory. Leaking which `.sst` file is visited would reveal which part of the database is being accessed. Therefore, the user should declare the entire `SST` directory as `oblivious`.



(a) A simple SQL database consisting of two randomly accessed database files, one sequentially accessed log file, and one database schema file.



(b) A simple key-value store consisting of one configuration file, one sequentially accessed log file and a directory consisting of multiple files that jointly store the database.

Figure 2: Motivating examples.

For files and directories labeled as `encrypted` or `oblivious`, PODS will additionally guarantee the integrity of the storage, thus preventing untrusted domains from tampering with trusted domain's data.

2.2.2 Design and Implementation of PODS

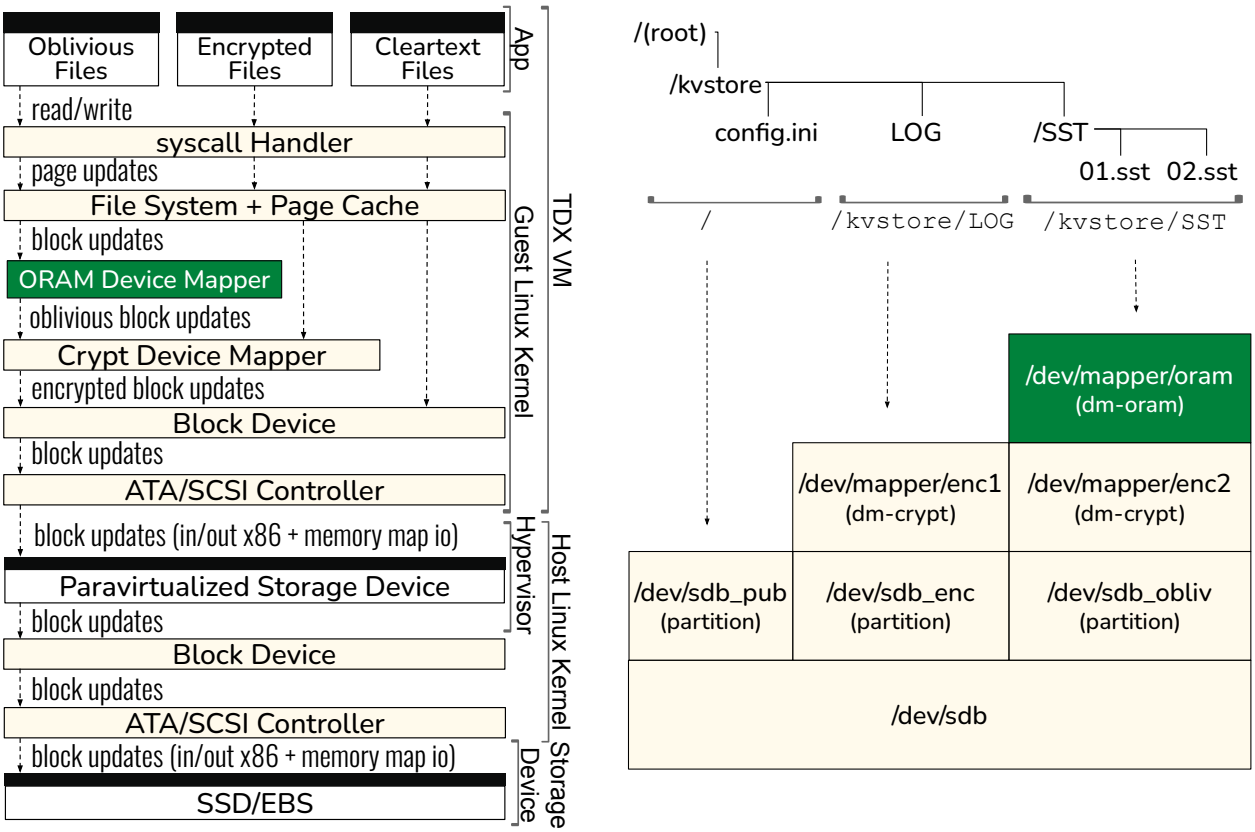
In Phase II, we will focus on developing PODS based on Intel's TDX technology, and bring it to market. TDX supports running legacy applications as a virtual machine inside the hardware enclave, offering a low-friction path towards adoption. The anticipated deliverables include 1) the source code and documentation of our PODS container, and 2) launching our PODS container on a major cloud platform such as Azure or Google Cloud. We now break this task down into a few key milestones.

Milestone 2.a: single ORAM for all oblivious files and directories. We will first implement a baseline version of PODS, where all files and directories labeled as `oblivious` will be mapped to this oblivious block device, and stored in a single ORAM data structure. When the trusted PODS container accesses this oblivious block device, all accesses will be protected, including the file/directory being accessed and the specific part of the file.

We depict the architecture of our PODS (baseline version) in Figure 3. The untrusted host OS provides a para-virtualized storage device abstraction to the trusted VM. Since anything written to the para-virtualized storage device can be observed by the host OS, the trusted domain is responsible for encrypting any sensitive blocks before they are written to the device. As shown in Figure 3, the OS kernel of the trusted domain includes a "crypt device mapper" (called `dm-crypt`) which intercepts sensitive reads and writes to the underlying block device, and handles the encryption and decryption.

In PODS, we will create an "ORAM device mapper" on top of the original "crypt device mapper" provided by the Linux kernel, as shown in green in Figure 3. Our ORAM device mapper will remap each *logical* block request to a small set of random-looking *physical* accesses [SvDS+13, WCS15]. For files that do not contain sensitive information, the ORAM and crypt device mappers may be bypassed for efficiency.

In Figure 3b, we illustrate the disk layout using one of the motivating examples described in Section 2.2.1. As mentioned earlier, we require that the child directory's security label be at least as strong as that of its parent directory. In this figure, the root directory denoted `/`, the directory `/kvstore`, and the `config.ini` file are labeled as `cleartext`. The `/kvstore/LOG` file is labeled as `encrypted`, and the `/kvstore/SST` directory is labeled as `oblivious`. The storage medium is divided into three partitions: a cleartext partition (`/dev/sdb_pub`), an encrypted partition (`/dev/sdb_enc`), and an oblivious partition (`/dev/sdb_obliv`). For the encrypted partition, the device mapper `/dev/mapper/enc1` handles the en-



(a) Architectural diagram

(b) Example file system and block device layout

Figure 3: Architecture of PODS

cryption and decryption of the data. For the oblivious partition, the device mapper `/dev/mapper/enc2` handles the encryption and decryption, and our ORAM device mapper named `/dev/mapper/oram` sits on top of `/dev/mapper/enc2`. In the file system, each directory is also treated as a file which stores the metadata associated with the directory (including information about child directories and files). Depending on the type annotations of directories or files, they are assigned to the appropriate partitions.

Milestone 2.b: separating into smaller ORAMs. So far, we have assumed that all oblivious files and directories are stored within the same ORAM, on a single oblivious block device. For latency-demanding applications such as real-time systems, we can further optimize performance by partitioning the files and directories across smaller ORAMs.

For example, in Figure 2a, suppose that database's root directory `sqldb` is marked with `cleartext`, and the two database files `faculty.db` and `student.db` are marked as `oblivious`. This security annotation suggests that while the system need not hide which file (i.e., `student.db` or `faculty.db`) is being accessed, it must conceal which record within these files is being accessed. In this case, we can separate `faculty.db` and `student.db` into two distinct ORAMs. Since an ORAM's access time grows logarithmically with the size of the ORAM (disregarding the recursion logarithm which is typically just 2 or 3 in practice), partitioning into two ORAMs results in faster query times. On the other hand, if the database's root directory `sqldb` is marked as `oblivious`, then all directories and files under `sqldb` must also be labeled as `oblivious` and share the same ORAM. In particular, marking a directory as `oblivious` implies that we need to hide which specific file within the directory is being accessed.

Separating the files and directories into different ORAMs will require us to architect PODS differently.

Rather than implementing the ORAM at the block device level, we will implement it at the file system level. Specifically, we will need to implement a file system module that maps the oblivious files or directories to separate ORAMs, with each ORAM stored its own file. This means that if a directory is labeled as `oblivious`, we will generate a single ORAM'ed file that stores all of its child files and subdirectories.

Milestone 2.c: Masking file sizes through differential obliviousness. Separating into smaller ORAMs promises better performance than having a single gigantic ORAM, but at the cost of slightly reduced security. Specifically, if the maximum size of a directory or file is not known at creation time, we may need to resize the files or directories dynamically during use. In this case, the host OS may infer that a resize operation is taking place by observing the access patterns to the para-virtualized storage. For example, if the trusted VM is accessing two contiguous regions in lock-step, this could indicate a resize operation where a file is copied to a new region. To mitigate such leakage, we plan to explore methods for introducing noise into the timing of the resize operations. To formally quantify the privacy of this approach, we will build upon our prior work which established a scientific foundation for "differential obliviousness (DO)" [CCMS22, ZSCM23, ZZCS24, QJS+22, CZSC21]. Specifically, DO applies the mathematical framework of differential privacy [DMNS06] to reason about access pattern privacy. In addition to masking the timing of the resize operations, DO also allows us to obfuscate the total number of accesses.

Anticipated performance. We anticipate the following ballpark performance for PODS. On a server-grade machine with SSD drive (e.g., Google Cloud Platform C3 instance, with 16-cores and 176 GB of RAM, and 50k iops), the estimated access time for a 4KB block in oblivious storage should is approximately 200-330μs for a 10MB ORAM, 270-820μs for a 10GB ORAM, and 330-1370μs for a 1TB ORAM. These estimates are based on our existing ORAM implementation [Labb], with adjustments to account for the overhead due to the crash recovery mechanism that we plan to implement.

Longer term: alternative realization based on SGX. After Phase II, we plan to develop an alternative implementation of PODS based on Intel's SGX, offering a slightly different tradeoff between the size of the Trusted Computing Base (TCB) and ease of deployment. Specifically, TDX typically runs the entire OS in the TEE, and thus OS becomes part of the trusted computing base (TCB). In comparison, with SGX, we typically run only the security-critical application inside the TEE, thus removing the OS from the TCB. We choose to prioritize TDX in Phase II partly because it is slightly easier to migrate legacy applications (along with the underlying OS) using TDX.

2.3 Oblivious STL

While ORAM provides a *generic* way to compile *any* program to an oblivious counterpart, in Oblivious STL, we design and develop *customized* oblivious implementations tailored to each computational task (or classes of computational tasks). This way, we can achieve asymptotical concrete performance improvements over generic ORAM [WNL+14, LWN+15, BSA13, ZE13, ACN+20, RS21, Shi20, JLS21, AHR24, AH25]. Therefore, Oblivious STL can be viewed as a privacy-preserving counterpart of the standard libraries that come with mainstream programming languages (e.g., C++'s STL and Rust's `std`).

As mentioned in Section 1, in Phase I, we developed optimized oblivious implementations of standard data structures such as ordered map, unordered map, array, queue, stack, priority queue, and common utility algorithms such as sorting, shuffling, filtering, and union find. In Phase II, the expected deliverable is an expanded and more optimized Oblivious STL library, supporting richer algorithmic building blocks. Specifically, we break down the proposed work into two major milestones.

Milestone 3.a: expand the coverage of the library. We plan to expand our Oblivious STL with richer primitives, including but not limited to data structures such as trie and range query data structures, set operations such as union and intersection, graph algorithms such as breadth-first search, depth-first search,

connectivity, Euler tour, tree contraction, shortest path, and other common computational tasks such as sketching algorithms, clustering algorithms, page rank, and matrix factorization.

Milestone 3.b: enclave-friendly adaptations and parallelism. One interesting research challenge is to adapt known oblivious algorithms [SvDS+13, WCS15, WNL+14, JLS21, Shi20, BSA13, GOT12, EGT10, ACN+20, ZE13] to enclave-friendly versions. Specifically, the standard literature [GO96, Gol87, SvDS+13, WCS15, Shi20, ACN+20] typically considers a theoretical architecture called the *word RAM* model, where computation time (in terms of the number of computation steps) is the sole performance metric. In practice, the TEE model of computation differs from the word RAM. Specifically, page swaps are a heavy-weight operation for TEEs: whenever the enclave wants to fetch some (encrypted) page from insecure memory, disk, or networked cache, it incurs heavy-weight operations including encryption and decryption, and possibly a context switch. This model of computation fits perfectly with the external-memory model that has been extensively studied in the algorithms literature [AV88, Dem02]. In some previous implementations [MPC+18, EZ19] that did not optimize the algorithms to the external-memory model, the page swap can take up 90% of the total running time or more. By contrast, we will optimize both the computation time and the page swap overhead. To achieve this we will borrow techniques from the external-memory algorithms literature [AV88, Dem02], which allow us to create a locality-friendly layout of our oblivious data structures as they are stored in memory and on disk. Besides enclave-friendly adaptations, we also plan to add multi-threading support to take advantage of the parallelism available on multi-core architectures.

Our oblivious STL library also serves as a foundation of our optimizing compiler thrust described in Section 2.4, since our compiler will emit to these lower-level oblivious building blocks and avoid generic ORAM whenever possible.

2.4 Optimizing Compiler and Oblivious Type System

As mentioned in our commercialization plan, one revenue stream comes from offering customized, high-performance oblivious implementations tailored to our customers' specific computational tasks. To make this effort scalable in terms of human expertise, we will make an optimizing compiler to semi-automate this task. With the optimizing compiler, our human experts only need to annotate our customers' code with certain hints, and our compiler will generate optimized implementations guided by such hints:

- *Security annotations.* The human expert will can label some input variables as `private` or `public`. Note that the human expert need not label all variables in the code, only the input variables to the program. Using standard information-flow analysis, our compiler will automatically infer the labels of the intermediate variables. These security annotations help the compiler infer which memory accesses are sensitive and must be protected using ORAM, and which ones are safe to reveal.
- *Obliviousness-friendly language constructs.* Certain programming paradigms can give the compiler hints on how to generate an optimized oblivious implementation. For example, parallel programming constructs such as map-reduce [LWN+15, GM11, CCLS20], fork-join [RS21], and the GraphLab model [NWI+15] are friendly to obliviousness — they can be compiled to primitives such as oblivious sorting and shuffling, without going through the generic ORAM compilation. Our earlier works [LWN+15, LHH+15] also introduced other language-level constructs that are obliviousness-friendly, and lead to one or more orders of magnitude performance improvement over naïvely applying generic ORAM.

In Phase II, we will extend the standard Rust language with the appropriate security labels and obliviousness-friendly annotations. We will then develop an optimizing compiler that consumes these hints provided by a human expert to generalize highly optimized target code. After Phase II, we plan to expand our effort and provide support for other popular languages. The reason to prioritize Rust is because our Phase I customer discovery efforts revealed that most of existing and potential customers are simultaneously performance- and

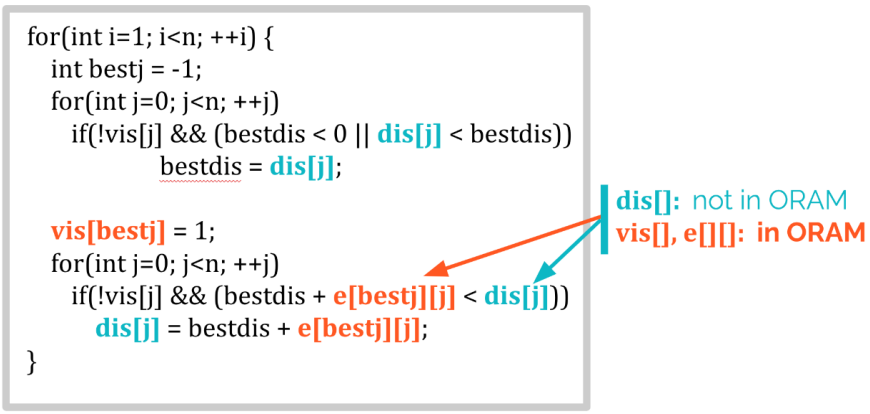


Figure 4: Snippet of Dijkstra's shortest path algorithm: the `dis[]` array need not be placed in ORAM, since it is always accessed sequentially. The `vis[]` and `e[][]` arrays must be placed in ORAM since their access patterns depend on the graph structure. Our compiler performs such partitioning through static analysis.

security-aware, and use Rust as their primary programming language, which offers a sweet spot between performance and type safety.

The anticipated deliverables include the specifications of the obliviousness-aware annotations on top of Rust, an optimizing compiler, as well as specifications of a memory-trace oblivious type system. We now break down this task into a few major milestones.

Milestone 4.a: partitioning of sensitive and non-sensitive accesses. The first milestone is to statically analyze a program, and partition the accesses into sensitive ones and non-sensitive ones. For example, Figure 4 shows a snippet of code from the Dijkstra's shortest path algorithm. In this program, the `dis[]` array is always accessed sequentially, and thus we only need to encrypt the array contents and need not place the array in ORAM. By contrast, the `vis[]` and `e[][]` arrays' access patterns depend on the graph structure, and thus we need to place these arrays in ORAMs. To make this kind of static analysis possible, we will introduce security labels into the Rust language, so the compiler is aware which input variables are sensitive. Using techniques described in our prior academic publications [LHS13, LHH+15, LWN+15], our compiler can then perform information flow tracking, infer the labels of intermediate variables, and determine whether accesses to an array leak secret information or not. Further, for arrays that must be placed in ORAMs, our compiler can also determine whether they can share the same ORAM, or can be placed in separate ORAMs. For example, suppose we encounter a secret `if`-conditional like the following:

```
if (cond):
        ans = x[t];         // t: labeled as private
else:
        ans = y[t];
```

If `cond` is labeled as private, then we should place `x[]` and `y[]` in the same ORAM, since which array is accessed leaks information about `cond`. Else, if `cond` is labeled as public, then it is okay to place `x[]` and `y[]` in separate ORAMs.

Milestone 4.b: compile to lower-level oblivious primitives. For large classes of computational tasks, we do not need to compile them to generic ORAM [SvDS+13, WCS15]. Compiling them to some lower-level algorithmic building blocks such as sorting, shuffling, or oblivious data structures result in significantly faster implementations. In our previous works [LWN+15, NWI+15, RS21, CCLS20, DLP+19], we have shown how to transform parallel algorithms implemented using of MapReduce, GraphLab, or fork-join syn-

tax directly to primitives such as oblivious sorting, shuffling, and segmented prefix sum. We have also shown how to transform inductive data structures directly to lower-level oblivious primitives rather than a full-fledged ORAM itself [LWN+15]. Further, subsequent research by others [AHR24, AH25] have also shown frameworks of algorithm design that are obliviousness-friendly, i.e., they can be converted to oblivious implementations without going through generic ORAM.

In the proposed work, we will explore how to convert these obliviousness-friendly algorithmic frameworks to language-level constructs. We will then enhance the Rust language with these language-level constructs. This way, a human expert can examine our customers' application code, and do some local rewriting to convert performance-critical portions of the code to use these obliviousness-friendly programming paradigms. Our compiler will then emit target code that avoids the use of generic ORAM whenever possible, leading to significantly speedups. Among these obliviousness-friendly algorithmic frameworks, some (e.g., MapReduce, GraphLab, fork-join, loop-coalescing) have already been well-established as programming abstractions in different contexts such as parallel computation; whereas some others (e.g., immutable memory [AH25], single-access machine [AHR24]) require additional research so that we can covert them to easy-to-use programming abstractions.

Milestone 4.c: type checking for application-level code. During our Phase I customer discovery efforts, some of our partners such as Signal and some blockchain companies expressed interest to have formally verified security for the oblivious implementations. To this end, we will take advantage of the theory of memory-trace oblivious type system [LHS13, LHH+15] demonstrated in our earlier research. Specifically, a memory-trace oblivious type system augments a traditional information-flow type system to be trace-aware. A traditional information flow type system prevents secret information from flowing into public variables which are observable to the adversary. For example, a variable whose value depends on the secret key should not be sent over a public network channel. On top of that, a memory-trace oblivious type system also treats memory access traces as being observable by the adversary. This way, it prevents secret information from flowing into not just public variables, but also memory traces.

To extend Rust with an oblivious type system and formally verify the obliviousness of a Rust program, we plan to proceed in two steps. The first step (i.e., milestone 4.c) is to treat our underlying oblivious primitives (including ORAM, and primitives provided by our Oblivious STL) as secure, and type check application-level code that leverages these oblivious primitives. Such an oblivious type system essentially guarantees that any well-typed program is formally verified to satisfy memory-trace obliviousness, as long as the underlying oblivious primitives are secure. The next step is to type check even the underlying oblivious primitives themselves, as described below as part of our longer-term goals after Phase II.

Longer-term: type checking for the underlying oblivious primitives. The next step is to enrich to type system so that we can even type check our own implementations of the underlying oblivious primitives. Because many of these oblivious primitives rely on randomness to ensure obliviousness, we would need to enhance our type system with probabilistic reasoning to accomplish this goal. The elegant work of Darais et al. [DSLH20] has already demonstrated the theoretical feasibility of such an probabilistic oblivious type system. After Phase II, we plan to explore how to make these techniques practical.

2.5 Longer Term Goal: Replacing TEE with Cryptography

Our Phase II efforts focuses on realizing oblivious computation based on Trusted Execution Environments (TEE). However, our longer-term vision is to remove the trust in the TEE by marrying oblivious computation with a "cryptography-enabled secure computer". In our academic research in CMU, we have identified two highly promising approaches towards this goal:

- **(Multi-party) Garbled RAM.** In garbled RAM, the TEE is replaced with a 2-party computation (2PC) or multi-party computation (MPC). The practical state-of-the-art for Garbled RAM (for 2PC) is our most

recent scheme called PicoGRAM [pic], which achieves roughly 30ms-40ms online time and 300ms-600ms total time per query for a large database of size 2^{30}. Importantly, unlike its more classical counterpart Garbled Circuits [Yao82], Garbled RAM allows secure accesses to large datasets without converting the entire dataset into a circuit, and then securely evaluting this linear-sized circuit.

- **PIR.** Private Information Retrieval (PIR) [CGKS95, CG97, BIM00, CK20, ZPSZ24, LMW23, HHCG$^+$23, MW22] allows clients to fetch data an untrusted database without disclosing their access patterns. Classical PIR schemes [CGKS95, CG97, HHCG$^+$23, MW22] that do not perform preprocessing are subject to having at least linear cost per query which makes them unscalable to big data settings. Fortunately, our prior research showed how to make PIR practical — by introducing a preprocessing phase [ZPSZ24], we can achieve sublinear cost per query. Our implementation (which represents the state of the art) achieves roughly 12ms online time and 40ms total time per query (plus an additional ping latency).

Garbled RAM and PIR each come with its own pros and cons. Garbled RAM can support general computation and naturally accommodates database updates; however, it requires a thresholded trust model involving two or more servers. By contrast, PIR has the advantage of requiring only a single untrusted server, but so far, its practical efficiency is known only for retrieval-only queries, and does not extend to generic computation, despite some initial theoretical feasibilities [LMW23].

In the longer term, we plan to explore how to adapt Garbled RAM or PIR to the high-value application scenarios discussed in this proposal, to remove the TEE from the trust.

3 Broader Impacts

Open-source tool chains for oblivious computation. Our project has already produced and will continue to produce open-source toolchains for oblivious computation. There is an urgent demand for an open-source toolchain like this. For example, partly because of the lack of a production-quality open-source library at the time, Signal took a detour and implemented a naïve batched linear scan algorithm back in 2017, which required 500 servers to handle their load. It was not until 2022 that they eventually upgraded their code to use the Path ORAM algorithm co-invented by our Chief Scientist Shi [SvDS$^+$13], thus reducing the number of servers to 6, saving almost a $100\times$ factor in computing bill.

Our new open-source toolchains will fill this urgent demand, and pave the way for the effortless adoption of oblivious computation for companies and organizations that wish to offer privacy to their users.

Industry consortium on oblivious computation. We also plan to form an industry consortium on oblivious computation. Today, early adopters like Signal and Meta have developed their own in-house implementation of oblivious computation. However, our implementation outperforms those of Signal and Meta by $8\times$ to $106\times$. As more industry leaders invest in oblivious computation, consolidating efforts through an open-source consortium is both timely and strategic. A key objective of the consortium is to unify R&D efforts and collaboratively contribute to the development, optimization, and maintenance of a shared open-source repository for oblivious computation. We have already discussed this initiative with our existing partners and received strong support. For example, Signal expressed interest to migrate to our code, and has shared with us the detailed dimensions of their dataset for performance validation. Our other existing and prospective partners including Ethereum, Flashbots, Meta, Worldcoin, Teleport, and Layer Zero are also natural participants in this consortium. We expect to welcome many more partners in Phase II.

A new eco-system for private data in AI. Wide-scale adoption of oblivious computation can help create a new eco-system for private data. Specifically, by providing strong privacy guarantees for users, we can incentivize users to contribute more private data. Business can in turn then take advantage of the more fine-grained data (e.g., user behavioral patterns, usage statistics, bug reports, etc.) to improve their data analytics

and AI results, thus offering better services to users. For example, in the past, by leveraging privacy-preserving data aggregation techniques [EPK14], Google was able to collect fine-grained user browsing behavior which they otherwise would have been reluctant to collect due to the sensitive nature of the data and the added accountability.

Oblivious computation can also enable a new private data market-place where users are in full control of their private data, and can securely contribute their data for data analytics tasks (e.g., clinical studies, population-wide studies). Meanwhile, businesses that wish to train machine learning models can provide incentives to users who contribute their data. Such an eco-system for private data can level the playing ground for small businesses, and help to mitigate the data-monopoly that we see today, where a few big companies control large amounts of user data.

Impact on other areas of confidential computing. Besides TEE-based solutions, several other companies [ros, zam, dua] are working to commercialize confidential computing solutions based on multi-party computation (MPC) or fully-homomorphic encryption (FHE). However, without the help of oblivious algorithms, these techniques require converting the entire database into a circuit and then securely evaluating the circuit, resulting in linear cost per query. For example, Signal estimated that performing a linear scan wrapped in efficient FHE results in more than 300M USD computing bill per year, which is simply unviable for them [rol].

The oblivious algorithms we develop will nicely complement these cryptographic based solutions for confidential computing. As explained in Section 2.5, oblivious algorithms allow us to transform traditional secure computation techniques, developed for the circuit model of computation, into the Random-Access-Machine (RAM) model, thus enabling secure accesses to large (encrypted or secret shared) datasets with only (poly-)logarithmic overhead. Therefore, our proposed work on oblivious algorithms is also *key to scaling cryptographic confidential computing to large data*. In fact, as mentioned in Section 2.5, our own longer-term agenda is also to remove the TEE and enable cryptography-based confidential computing on big data.

Societal impact. Our efforts of promoting oblivious computation promise wide societal impact. As mentioned, once industry leaders such as Signal and Ethereum adopt our technology, we plan to leverage these success stories to raise awareness, and help establish oblivious computation as a new industry standard.

Because oblivious computation is lightweight and supports general-purpose computation, we aim to advance a future where privacy is "default-on," replacing today's "default-off" norm. This will make the cyber-space significantly safer for ordinary users, and empower them to control their private data. Further, by addressing the privacy concerns, we can foster greater willingness to share data, thus unlocking new opportunities for utility and social good through population-scale machine learning and data mining.

References

[1rp] Automata 1rpc. https://docs.1rpc.io/.

[ACN+20] Gilad Asharov, T-H. Hubert Chan, Kartik Nayak, Rafael Pass, Ling Ren, and Elaine Shi. Bucket oblivious sort: An extremely simple oblivious sort. In *SOSA*, 2020.

[AdBPB+23] David W. Archer, Borja de Balle Pigem, Dan Bogdanov, Mark Craddock, Adria Gascon, Ronald Jansen, Matjaz Jug, Kim Laine, Robert McLellan, Olga Ohrimenko, Mariana Raykova, Andrew Trask, and Simon Wardley. UN handbook on privacy-preserving computation techniques, 2023.

[AH25] Ananya Appan and David Heath. Oblivious immutable memory. *IACR Cryptol. ePrint Arch.*, page 593, 2025.

[AHR24] Ananya Appan, David Heath, and Ling Ren. Oblivious single access machines - a new model for oblivious computation. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 3080–3094, 2024.

[alca] Alchemy. https://www.alchemy.com/.

[alcb] Alchemy pricing. https://www.alchemy.com/pricing.

[Ant25] Anthropic. Confidential inference systems: Design principles and security risks. https://assets.anthropic.com/m/c52125297b85a42/original/Confidential_Inference_Paper.pdf, 2025.

[app] Private cloud compute: A new frontier for ai privacy in the cloud. https://security.apple.com/blog/private-cloud-compute/.

[ATG+16] Sergei Arnautov, Bohdan Trach, Franz Gregor, Thomas Knauth, Andre Martin, Christian Priebe, Joshua Lind, Divya Muthukumaran, Dan O'Keeffe, Mark L. Stillwell, David Goltzsche, David Eyers, Rüdiger Kapitza, Peter Pietzuch, and Christof Fetzer. Scone: secure linux containers with intel sgx. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, OSDI'16, page 689–703, USA, 2016. USENIX Association.

[AV88] Alok Aggarwal and S. Vitter, Jeffrey. The Input/Output Complexity of Sorting and Related Problems. *Commun. ACM*, 31(9):1116–1127, September 1988.

[aws] Amazon elastic container service. https://aws.amazon.com/ecs/.

[azua] Azure ai confidential inferencing: Technical deep-dive. https://techcommunity.microsoft.com/blog/azureconfidentialcomputingblog/azure-ai-confidential-inferencing-technical-deep-dive/4253150.

[azub] Containers on azure. https://azure.microsoft.com/en-us/products/category/containers#Products.

[BIM00] Amos Beimel, Yuval Ishai, and Tal Malkin. Reducing the servers computation in private information retrieval: Pir with preprocessing. In *CRYPTO*, pages 55–73, 2000.

[bra20] Themis: Towards a decentralized ad platform with reporting integrity. https://brave.com/themis/, 2020.

[BSA13] Marina Blanton, Aaron Steele, and Mehrdad Alisagari. Data-oblivious graph algorithms for secure computation and outsourcing. In *ASIA CCS*, 2013.

[bui] Buildernet by flashbots. https://buildernet.org/.

[But] Vitalik Buterin. A maximally simple l1 privacy roadmap. https://ethereum-magicians.org/t/a-maximally-simple-l1-privacy-roadmap/23459.

[ccc] https://www.fortunebusinessinsights.com/confidential-computing-market-107794.

[CCLS20] T.-H. Hubert Chan, Kai-Min Chung, Wei-Kai Lin, and Elaine Shi. MPC for MPC: secure computation on a massively parallel computing architecture. In *11th Innovations in Theoretical Computer Science Conference, ITCS 2020, January 12-14, 2020, Seattle, Washington, USA*, volume 151 of *LIPIcs*, pages 75:1–75:52. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.

[CCMS22] T.-H. Hubert Chan, Kai-Min Chung, Bruce M. Maggs, and Elaine Shi. Foundations of differentially oblivious algorithms. *J. ACM*, 69(4):27:1–27:49, 2022.

[CG97] Benny Chor and Niv Gilboa. Computationally private information retrieval. In *STOC*, 1997.

[CGKS95] Benny Chor, Oded Goldreich, Eyal Kushilevitz, and Madhu Sudan. Private information retrieval. In *FOCS*, 1995.

[CK20] Henry Corrigan-Gibbs and Dmitry Kogan. Private information retrieval with sublinear online time. In *EUROCRYPT*, 2020.

[cona] https://www.businessresearchinsights.com/market-reports/container-technology-market-106710.

[Conb] Deirdre Connolly. Oblivious message retrieval. https://zfnd.org/oblivious-message-retrieval/.

[CS17] T.-H. Hubert Chan and Elaine Shi. Circuit OPRAM: unifying statistically and computationally secure orams and oprams. In *Theory of Cryptography - 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12-15, 2017, Proceedings, Part II*, volume 10678 of *Lecture Notes in Computer Science*, pages 72–107. Springer, 2017.

[CSS10] T.-H. Hubert Chan, Elaine Shi, and Dawn Song. Private and continual release of statistics. In *ICALP*, 2010.

[CSS11] T.-H. Hubert Chan, Elaine Shi, and Dawn Song. Private and continual release of statistics. *TISSEC*, 14(3):26, 2011.

[CZSC21] Shumo Chu, Danyang Zhuo, Elaine Shi, and T.-H. Hubert Chan. Differentially oblivious database joins: Overcoming the worst-case curse of fully oblivious algorithms. In *2nd Conference on Information-Theoretic Cryptography, ITC 2021, July 23-26, 2021, Virtual Conference*, volume 199 of *LIPIcs*, pages 19:1–19:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021.

[dbea] https://www.marketresearchfuture.com/reports/database-encryption-market-1801.

[dbeb] https://www.linkedin.com/pulse/database-encryption-market-forecast2024-2030-akhila-

[Dem02] Erik D. Demaine. Cache-oblivious algorithms and data structures. In *Lecture Notes from the EEF Summer School on Massive Data Sets*. BRICS, University of Aarhus, Denmark, June 27–July 1 2002.

[dev] Dod enterprise devsecops strategy guide. `https://dodcio.defense.gov/Portals/0/Documents/Library/DoDEnterpriseDevSecOpsStrategyGuide.pdf`.

[DFD+21] Emma Dauterman, Vivian Fang, Ioannis Demertzis, Natacha Crooks, and Raluca Ada Popa. Snoopy: Surpassing the scalability bottleneck of oblivious storage. Cryptology ePrint Archive, Paper 2021/1280, 2021. `https://eprint.iacr.org/2021/1280`.

[DLP+19] Dana Dachman-Soled, Chang Liu, Charalampos Papamanthou, Elaine Shi, and Uzi Vishkin. Oblivious network RAM and leveraging parallelism to achieve obliviousness. *J. Cryptol.*, 32(3):941–972, 2019.

[DMNS06] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *Theoretical Cryptography Conference (TCC)*, 2006.

[doc] Docker: Build, ship and run any app, anywhere. `https://www.docker.com/`.

[DPX+21] Nathan Daughety, Marcus Pendleton, Shouhuai Xu, Laurent Njilla, and John Franco. vcds: A virtualized cross domain solution architecture. In *MILCOM*, 2021.

[DSLH20] David Darais, Ian Sweet, Chang Liu, and Michael Hicks. A language for probabilistically oblivious computation. *Proc. ACM Program. Lang.*, 4(POPL):50:1–50:31, 2020.

[dua] Duality tech. `https://dualitytech.com/`.

[EGT10] David Eppstein, Michael T. Goodrich, and Roberto Tamassia. Privacy-preserving data-oblivious geometric algorithms for geographic data. In *GIS*, 2010.

[eip] EIP-1186: RPC-method to get Merkle proofs - eth_getproof. `https://eips.ethereum.org/EIPS/eip-1186`.

[EPK14] Úlfar Erlingsson, Vasyl Pihur, and Aleksandra Korolova. Rappor: Randomized aggregatable privacy-preserving ordinal response. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, CCS '14, page 1054–1067, 2014.

[EZ19] Saba Eskandarian and Matei Zaharia. Oblidb: Oblivious query processing for secure databases. *Proc. VLDB Endow.*, 13(2):169–183, oct 2019.

[fle25] Flexway o-sort: Enclave-friendly and optimal oblivious sorting. In *Usenix Security*, 2025.

[FMN20] Olivier Flauzac, Fabien Mauhourat, and Florent Nolot. A review of native container security for running applications. *Procedia Computer Science*, 2020. MobiSPC, FNC, International Conference on Sustainable Energy Information Technology.

[GM11] Michael T. Goodrich and Michael Mitzenmacher. Privacy-preserving access of outsourced data via oblivious RAM simulation. In *ICALP*, 2011.

[GO96] Oded Goldreich and Rafail Ostrovsky. Software protection and simulation on oblivious RAMs. *J. ACM*, 1996.

[Gol87] O. Goldreich. Towards a theory of software protection and simulation by oblivious RAMs. In *STOC*, 1987.

[gooa] Containers on google cloud. https://cloud.google.com/containers.

[goob] Google cloud pricing. https://cloud.google.com/pricing.

[gooc] Privacy-preserving smart input with gboard. https://security.googleblog.com/2020/10/privacy-preserving-smart-input-with.html.

[GOT12] Michael T. Goodrich, Olga Ohrimenko, and Roberto Tamassia. Data-oblivious graph drawing model and algorithms. *CoRR*, abs/1209.0756, 2012.

[gsc] gsc — gramine shielded containers. https://gramine.readthedocs.io/projects/gsc/en/latest/.

[GTRS25] Tianyao Gu, Afonso Tinoco, Sri Harish G Rajan, and Elaine Shi. Picogram: Practical garbled ram from decisional diffie-hellman. In *CRYPTO*, 2025.

[gvi] gvisor. https://gvisor.dev/.

[HHCG+23] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. One server for the price of two: Simple and fast single-server private information retrieval. In *Usenix Security*, 2023.

[infa] Consensys: Over 350k web3 developers now use blockchain development platform infura. https://consensys.io/blog/over-350k-web3-developers-now-use-blockchain-development-platform-infura.

[infb] Dialogue with infura co-founder: Why we plan to launch a decentralized protocol in 2023. https://www.chaincatcher.com/en/article/2080321.

[infc] Infura. https://www.infura.io/.

[infd] Infura pricing. https://www.infura.io/pricing.

[JFHS22] Sara Jordan, Clara Fontaine, and Rachele Hendricks-Sturrup. Selecting privacy-enhancing technologies for managing health data use. *Frontiers in Public Health*, 10, 2022.

[JLS21] Zahra Jafargholi, Kasper Green Larsen, and Mark Simkin. Optimal oblivious priority queues. In Dániel Marx, editor, *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms, SODA 2021, Virtual Conference, January 10 - 13, 2021*, pages 2366–2383. SIAM, 2021.

[Laba] Oblivious Labs. Oblivious evm op codes. https://github.com/obliviouslabs/revm-oo.

[Labb] Oblivious Labs. Oblivious key-value store by oblivious labs. https://github.com/obliviouslabs/oram.

[Labc] Oblivious Labs. Open source code: oblivious balance checker for erc20 tokens. https://github.com/obliviouslabs/oblivious-erc20-state.

[Labd] Oblivious Labs. Reproducible benchmarks for oblivious computation. https://github.com/obliviouslabs/benchmarks.

[Labe] Oblivious Labs. Wbtc balance checker demo. https://obliviouslabs.com/WBTCdemo/.

[LHH+15] Chang Liu, Michael Hicks, Austin Harris, Mohit Tiwari, Martin Maas, and Elaine Shi. Ghostrider: A hardware-software system for memory trace oblivious computation. In *ASPLOS*, 2015.

[LHS13] Chang Liu, Michael Hicks, and Elaine Shi. Memory trace oblivious program execution. CSF '13, pages 51–65, 2013.

[LMW23] Wei-Kai Lin, Ethan Mook, and Daniel Wichs. Doubly efficient private information retrieval and fully homomorphic ram computation from ring lwe. In *STOC*, 2023.

[LSX19] Wei-Kai Lin, Elaine Shi, and Tiancheng Xie. Can we overcome the $n \log n$ barrier for oblivious sorting? In *SODA*, 2019.

[LWN+15] Chang Liu, Xiao Shaun Wang, Kartik Nayak, Yan Huang, and Elaine Shi. Oblivm: A programming framework for secure computation. In *IEEE Symposium on Security and Privacy*, 2015.

[lxc] lxc linux containers. http://lxc.sourceforge.net/.

[lxd] Linux container daemon (lxd). https://linuxcontainers.org/lxd/.

[met] Meta's oram implementation. https://github.com/facebook/oram.

[mon] Mongodb data encryption. https://www.mongodb.com/products/capabilities/security/encryption.

[mpc] Global secure multiparty computation market dynamics. https://www.marketsandmarkets.com/Market-Reports/secure-multiparty-computation-market-67797344.html.

[MPC+18] Pratyush Mishra, Rishabh Poddar, Jerry Chen, Alessandro Chiesa, and Raluca Ada Popa. Oblix: An efficient oblivious search index. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 279–296, 2018.

[MW22] Samir Jordan Menon and David J. Wu. SPIRAL: Fast, high-rate single-server PIR via FHE composition. In *IEEE S&P*, 2022.

[nit] Aws nitro enclaves. https://aws.amazon.com/ec2/nitro/nitro-enclaves/.

[NWI+15] Kartik Nayak, Xiao Shaun Wang, Stratis Ioannidis, Udi Weinsberg, Nina Taft, and Elaine Shi. Graphsc: Parallel secure computation made easy. In *2015 IEEE Symposium on Security and Privacy*, pages 377–394, 2015.

[ope] https://openrouter.ai/.

[OR] Timon Van Overveldt and Daniel Ramage. Discovering new words with confidential federated analytics. https://research.google/blog/discovering-new-words-with-confidential-federated-analytics/.

[ost] Oblivious stl library. https://github.com/obliviouslabs/rostl.

[Ost90] R. Ostrovsky. Efficient computation on oblivious RAMs. In *ACM Symposium on Theory of Computing (STOC)*, 1990.

[PAD+23] Antigoni Polychroniadou, Gilad Asharov, Benjamin Diamond, Tucker Balch, Hans Buehler, Richard Hua, Suwen Gu, Greg Gimler, and Manuela Veloso. Prime match: a privacy-preserving inventory matching system. In *Proceedings of the 32nd USENIX Conference on Security Symposium*, SEC '23. USENIX Association, 2023.

[pic] Picogram: an efficient garbled ram scheme. Open-source code released at `https://github.com/picogramimpl/`.

[pri] Private communication with Rolfe Schmidt, tech-lead at signal.

[PRZB12] Raluca Ada Popa, Catherine M. S. Redfield, Nickolai Zeldovich, and Hari Balakrishnan. Cryptdb: processing queries on an encrypted database. *Commun. ACM*, 55(9):103–111, September 2012.

[QJS+22] Lianke Qin, Rajesh Jayaram, Elaine Shi, Zhao Song, Danyang Zhuo, and Shumo Chu. Differentially oblivious relational database operators. *Proc. VLDB Endow.*, 16(4):842–855, 2022.

[qui] Quick node. `https://www.quicknode.com/`.

[RL] Polaris Market Research and Consulting LLP. Confidential computing market set to expand at a staggering 63.1% cagr, reaching usd 417.18 billion by 2032. `https://finance.yahoo.com/news/confidential-computing-market-set-expand-105100687.html`.

[rol] Talk at confidential computing summit 2023 by signal tech lead rolfe schmidt.

[ros] Roseman labs. `https://rosemanlabs.com/`.

[RS21] Vijaya Ramachandran and Elaine Shi. Data oblivious algorithms for multicores. In *SPAA '21: 33rd ACM Symposium on Parallelism in Algorithms and Architectures, Virtual Event, USA, 6-8 July, 2021*, pages 373–384. ACM, 2021.

[SAD19] Sari Sultan, Imtiaz Ahmad, and Tassos Dimitriou. Container security: Issues, challenges, and the road ahead. *IEEE Access*, 7:52976–52996, 2019.

[SCSL11] Elaine Shi, T.-H. Hubert Chan, Emil Stefanov, and Mingfei Li. Oblivious RAM with $O((\log N)^3)$ worst-case cost. In *ASIACRYPT*, 2011.

[Shi20] Elaine Shi. Path oblivious heap: Optimal and practical oblivious priority queue. In *2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18-21, 2020*, pages 842–858. IEEE, 2020.

[siga] Signal's oblivious ram implementation. `https://github.com/signalapp/ContactDiscoveryService-Icelake`.

[sigb] Technology deep dive: Building a faster oram layer for enclaves. `https://signal.org/blog/building-faster-oram/`.

[SNJC10] Elaine Shi, Yuan Niu, Markus Jakobsson, and Richard Chow. Implicit authentication through learning user behavior. In *Proceedings of the 13th International Conference on Information Security*, ISC'10, page 99–113, Berlin, Heidelberg, 2010. Springer-Verlag.

[SvDS+13] Emil Stefanov, Marten van Dijk, Elaine Shi, Christopher Fletcher, Ling Ren, Xiangyao Yu, and Srinivas Devadas. Path ORAM – an extremely simple oblivious ram protocol. In *CCS*, 2013.

[tde] 3 introduction to transparent data encryption. https://docs.oracle.com/en/database/oracle/oracle-database/19/asoag/introduction-to-transparent-data-encryption.html.

[TGS23] Afonso Tinoco, Sixiang Gao, and Elaine Shi. Enigmap: External-memory oblivious map for secure enclaves. In *32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9-11, 2023*, pages 4033–4050. USENIX Association, 2023.

[TGSM25] Afonso Tinoco, Tianyao Gu, Elaine Shi, and Andrew Miller. Scalable oblivious accesses to blockchain data. https://writings.flashbots.net/scalable-oblivious-accesses-to-blockchain-data, 2025.

[WCS15] Xiao Wang, T.-H. Hubert Chan, and Elaine Shi. Circuit ORAM: on tightness of the goldreich-ostrovsky lower bound. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, Denver, CO, USA, October 12-16, 2015*, pages 850–861. ACM, 2015.

[WNL+14] Xiao Shaun Wang, Kartik Nayak, Chang Liu, T-H. Hubert Chan, Elaine Shi, Emil Stefanov, and Yan Huang. Oblivious Data Structures. In *CCS*, 2014.

[Yao82] Andrew Chi-Chih Yao. Protocols for secure computations (extended abstract). In *FOCS*, 1982.

[YNZ25] Sen Yang, Kartik Nayak, and Fan Zhang. Decentralization of Ethereum's Builder Market. In *IEEE Symposium on Security and Privacy (SP)*, pages 1512–1530, Los Alamitos, CA, USA, May 2025. IEEE Computer Society.

[zam] Zama. https://www.zama.ai/.

[ZE13] Samee Zahur and David Evans. Circuit Structures for Improving Efficiency of Security and Privacy Tools. In *S & P*, 2013.

[ZPSZ24] Mingxun Zhou, Andrew Park, Elaine Shi, and Wenting Zheng. Piano: Extremely simple, single-server pir with sublinear server computation. In *IEEE S& P*, 2024.

[ZRX+23] Yuanbo Zhang, Daniel Ramage, Zheng Xu, Yanxiang Zhang, Shumin Zhai, and Peter Kairouz. Private federated learning in gboard, 2023.

[ZSCM23] Mingxun Zhou, Elaine Shi, T.-H. Hubert Chan, and Shir Maimon. A theory of composition for differential obliviousness. In *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part III*, volume 14006 of *Lecture Notes in Computer Science*, pages 3–34. Springer, 2023.

[ZZCS24] Mingxun Zhou, Mengshi Zhao, T.-H. Hubert Chan, and Elaine Shi. Advanced composition theorems for differential obliviousness. In *15th Innovations in Theoretical Computer Science Conference, ITCS 2024, January 30 to February 2, 2024, Berkeley, CA, USA*, volume 287 of *LIPIcs*, pages 103:1–103:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.

Data Management and Sharing Plan

All data generated in this NSF SBIR/STTR Phase II project is considered proprietary.

More specifically, we expect to use the following types of data during this Phase II project:

- *Public datasets.* Most of our blockchain customers care about providing oblivious accesses to public datasets, typically the up-to-date states and transactional logs of public blockchains. Such datasets need not be protected.

- *Synthetic datasets from customers/partners.* The performance of our ENCORE system depends only on the dimensions of the data (e.g., record size and the number of records), but does not depend on the actual data contents. Therefore, some of our customers and partners may send us a synthetic dataset that is at the same size and scale of their real dataset, for us to take performance measurements. For example, Signal, who is interested in migrating to our ENCORE libraries, has sent us a synthetic copy of their user database but populated with junk data.

  For such synthetic datasets, based on the customer's specific requirements, we may or may not be obliged to keep the dimensions of the database confidential. We will explicitly ask the customer to clarify whether we can use the database's dimensions in our white-papers and marketing materials.

- *Customer data.* Some of our customers may ask us to support the roll-out of our ENCORE technology by migrating their existing insecure database to ENCORE. We anticipate that some of these customers may ask us to sign non-disclosure agreements (NDAs) to keep their datasets confidential.

Facilities, Equipment and Other Resources

Facilities

Our company is located at 6425 Living Place, Suite 200 Pittsburgh 15206. All current team members (Afonso Tinoco, Elaine Shi, and Tianyao Gu) will be residing in Pittsburgh, PA during the performance period. Our advisor Andrew Miller (no-cost senior personnel) resides in Illinois. Currently, we work from home to conduct the R&D work; moreover, we collaborate closely through a combination of zoom meetings (e.g., joint with our remote advisor Andrew Miller) and in-person meetings in Pittsburgh. We will also rent some co-working space in Pittsburgh in the near future when our team grows.

During the performance period, we plan to hire 1 to 2 research engineer(s) for the proposed project. When this happens, we plan to rent (by the hour) a small co-working space in Pittsburgh. Part of the budget is reserved for this purpose.

Equipment

The company will provide computing support (e.g., laptops and desktop machines with TEE support) to conduct the proposed R&D work. Additionally, we also plan to run some experiments and evaluations using cloud services including Google Cloud, Amazon AWS and Microsoft Azure.

Commercialization Plan

A Market Opportunity

A.1 Our Products, Services, and Target Customers

Our ENCORE oblivious computation technology offers a general-purpose and high-performance confidential computing platform. ENCORE differentiates itself from most existing confidential computing technologies by enabling oblivious accesses to large datasets, thus protecting users' queries and intentions.

Software products. Our ENCORE framework encompasses a suite of software products, as described below:

1. **PODS oblivious container.** PODS is a new container technology that differentiates itself from existing containers by offering an oblivious storage backend, such that the sensitive application running in trusted domain can avoid leaking secret information through its access patterns to data. PODS will be made available both through licensing and through the marketplaces of major cloud platforms.
2. **Oblivious databases.** We will offer an oblivious key-value store to our customers. Unlike existing encrypted databases which protect only the data contents, our oblivious databases additionally protect the users' queries and intentions, by provably obfuscating the users' access patterns to data. In the longer term, we will also develop an oblivious SQL database and offer it to our customers.

Services. We plan to offer the following services.

1. **Oblivious accesses to public databases.** We plan to host an RPC service that provides oblivious accesses to public databases. Users and applications can query the public database while hiding their queries and intentions. Specifically, when the public database is blockchain data (e.g., Ethereum's state), our service can be viewed a privacy-enhanced version of today's RPC nodes such as Infura and Alchemy.
2. **Oblivious computation as a service.** For our customers that rely on our oblivious databases or containers, we will offer the following type of services:
   (a) **Customized optimizations**: We will provide customized optimizations tailored for our customers' computation needs. Specifically, as part of the proposed Phase II work, we will develop an optimizing compiler capable of automatically synthesizing an optimized oblivious implementation given a user-supplied source program along with certain obliviousness-aware annotations.
   (b) **Engineering support and consultation**: We will provide the following type of support and consultation to our customers: 1) assistance with migrating legacy services to using our ENCORE technologies; 2) support for software maintenance, updates, and horizontal scaling; 3) performance profiling and optimizations tailored for the customers' specific workloads.
   (c) **Security audit and review**: We will assist customers in reviewing their legacy services to identify privacy threats, and educate them about the benefits and performance implications of adopting oblivious computation. After they adopt oblivious computation, we provide security audits of their oblivious implementations using the memory-trace oblivious type system and static analysis techniques developed in Phase II.

Target customers. We target primarily enterprise customers, with the exception of our "oblivious accesses to public databases" services which may attract both businesses and end users alike. We anticipate that the strongest demand for our technology will come from the *AI, cloud computing, and blockchain* sectors. Within these sectors, we will focus on acquiring the following types of customers:

1. *Service providers that currently do not offer privacy but wish to integrate privacy into their existing services*. Our new partner Etheurem falls into this category — see Section A.3 for more details.

2. *Service providers that already offer privacy in their services, and wish to overcome scaling challenges and/or save computing costs.* For example, a company currently offering an FHE-based privacy solution will likely encounter scaling challenges as they acquire more customers, and would like to adopt oblivious computation to meet their scaling needs. Our partnerships with Signal is also of this nature since we can help them improve performance and cut computing costs.
3. *Service providers that currently offer only partial privacy, and wish to upgrade to full privacy*. Our collaboration with WorldCoin is of this nature — see Section A.3 for more details.
4. *Enterprises seeking to perform private collaborative AI with other organizations, or requiring oblivious access to public databases.* Examples include retail businesses and traditional financial institutions that want to unlock new value from their existing data through collaborative AI, as well as traders who need access to blockchain or public financial data while protecting their strategic intentions.

Our primary focus is on enterprises in North America. Currently, most of our existing and prospective partners are either incorporated in the US or Canada, or are international corporations with registered entities in the US.

A.2 Market Drivers

We now describe the market drivers for the adoption of oblivious computation.

A.2.1 Private AI and Cloud Computing

Market driver 1: increasing privacy awareness and demand. As AI and Large Language Models (LLMs) become increasingly integrated into our daily lives, privacy concerns have also intensified. These concerns arise from both sides — the services providers and the consumers of AI alike:

- On one hand, users and businesses that consume AI want to train customized models based on their private data; but are unwilling to share the data with AI service providers.
- On the other hand, services providers are concerned about privacy for two reasons. First, AI providers (e.g., Anthropic) want to license models to enterprise customers but are concerned about releasing their model weights which represent their core intellectual property. Second, without a private computation platform, service providers may hesitate to track fine-grained user behavior for fear of privacy backlash.

Market driver 2: privacy regulations and best practice recommendations. Another market driver for confidential computing comes from privacy regulations such as GDPR or HIPPA. Typically, these privacy regulations (effectively) require de-identification and removal of personally identifiable information during data sharing. While they do not make data encryption mandatory, they nonetheless strongly recommend adopting best practices such as data encryption, and "state-of-the-art" privacy techniques. In recent years, new recommendations have emerged on best practices for regulation-compliant information sharing and management (e.g., Jordan et al.[JFHS22] and the United Nations guidelines[AdBPB$^+$23]). In particular, the recommendations include cryptographic techniques such as multi-party computation (MPC) — our ENCORE framework can be viewed as a sigificantly more efficient alternative to MPC.

Market driver 3: unlocking new functionalities and improving the quality of AI recommendations. Exactly because of the rising privacy concerns and the existence of privacy regulations, integrating privacy into AI training and inferences promises high value. Satisfactorily addressing the users' and businesses' privacy concerns encourages them to more readily share data. This in turn enables safe and effective collaborative data analytics, unlocking new functionality from existing datasets. Increased data sharing also paves the way for higher-fidelity models and more personalized recommendations

Market driver 4: lower bar of entry for small players. Many startup companies that are part of the AI ecosystem want to gain the users' trust to successfully onboard users. For example, OpenRouter [ope] is a startup company providing an AI model aggregator — in essence, a one-stop marketplace and routing gateway for hundreds of large language models (LLMs). They want to offer users the conveniene of interacting with a set of LLM models using a unified interface, but without exposing users to more privacy risk than with interacting with the underlying LLMs directly. Specifically, they rely on confidential computing technology and promise users that OpenRouter itself cannot see the users' queries, even though the queries are eventually routed to a set of trusted LLM engines of the user's choice (e.g., those provided by Open AI, Anthropic, Google).

Market driver 5: increasing market competition. Recently, industry leaders including Apple [app], Google [OR], Microsoft [azua], Anthropic [Ant25] have raced to announce their commitment to delivering a new generation of privacy-focused AI services. The universal buy-in from major stake-holders will inevitably drive competition, which helps to establish privacy as a new industry norm.

Various companies have also rolled out privacy-preserving services, raising users' expectation and appetite for privacy. For example, Signal offers encrypted messenging and private contact discovery [sigb]; the Brave privacy-preserving browser offers privacy-preserving advertising [bra20], and so on. Financial institutions such as JP Morgan have developed and deployed privacy-preserving market-maker platforms [PAD+23]. Google has made continuous efforts towards privacy-preserving federated learning — in fact, the stochastic gradient descent algorithm in their Spanish GBoard [gooc] actually builds on senior personnel Shi's earlier privacy-preserving algorithms [CSS10, CSS11].

A.2.2 Blockchains

Market driver 6: lack of privacy leads to monetary loss for users. In the blockchain space, users' privacy concerns may even be driven by economic factors. Today, transactions are submitted in the clear, and block producers can easily front-run and back-run users' transactions to make risk-free profit — a phenomenon commonly referred to as Miner Extractable Value (MEV). Because victim users suffer from financial loss due to such MEV-style attacks, many would desire a private order flow architecture where their transactions are protected until the block is built and confirmed in the blockchain.

As users become increasingly aware of the financial risks posed by MEV attacks, blockchain service providers are now racing to incorporate privacy features into their offerings to retain user trust and loyalty. Meanwhile, initiatives by major players such as Ethereum [But] and Flashbots [TGSM25] to integrate privacy at the infrastructure level have also accelerated this paradigm shift towards privacy.

Market driver 7: privacy as the underpinning of decentralization. In the blockchain space, the push for privacy comes not just from the users, but also from the blockchain creators. Today, the aforementioned MEV-style attacks is causing the blockchain's eco-system to become increasingly centralized. Block builders nowadays offer private contracts to ordinary users and arbitragers alike, selling them favorable positions inside the block. Such offchain private contracts undermine the decentralized nature of the blockchain's ecosystem. A recent empirical study shows that more than 85% of Ethereum's blocks are created by the largest two block builders [YNZ25]. During Phase I, we engaged in close conversations with blockchain creators (e.g., Ethereum founder Vitalik Buterin) and major stake-holders such as Flashbots. Adding privacy to the order submission and processing infrastructure is their top priority, as it is critical for preserving the decentralized nature of the system — the very core of blockchain's value proposition.

A.2.3 Demand for Oblivious Computation

Market driver 8: existing confidential computing offerings fail to address emerging needs. In our customery discovery interviews, we identified various application domains that are not adequately served by the old generation of confidential computing technologies. Traditional approaches — such as disk or memory encryption and encrypted databases — focus on protecting data contents, but not the users' queries or intentions. However, in the emerging use cases described in Section A.3, it is precisely the users' queries or intentions that must be protected, whereas the underlying database is often public or already known to the service provider. This critical need is also reaffirmed by in the support letters from our partners, including Signal, Ethereum, Flashbots, and Worldcoin.

Market driver 9: oblivious computation as a complement or scaling solution for existing privacy offerings. While seemingly competing solutions — such as multi-party computation (MPC) and fully homomorphic encryption (FHE) — exist in the market, these technologies currently support only small-scale applications. In particular, without the use of oblivious computation techniques, both MPC or FHE would require securely evaluating a linear-sized circuit that encodes the entire dataset per memory access! As a result, these privacy offerings face inherent performance bottlenecks that become prohibitive as applications scale and data volumes increase. As described in Section 2.5 of our technical proposal, part of our long-term roadmap is to integrate oblivious computation techniques into MPC, enabling it to scale effectively to big data workloads.

As described in more detail in Section A.3, our Phase I commercialization efforts also uncovered compelling use cases where cryptography (e.g., zero-knowledge proofs) alone offers only partial privacy protection for users. In these scenarios, oblivious computation serves as the ideal complement to existing privacy-preserving technologies, enabling true, end-to-end privacy for users.

A.3 Some Concrete Use Cases

Below are some concrete use cases from our existing and new partners and customers.

- *Signal.* As mentioned, Signal already employs ORAM and TEE (short for Trusted Execution Environment) as the backbone of their private contact discovery [sigb]. Private contact discovery is a representative application involving a large database, such that access patterns to the database directly leak the users' secret contact information. Before rolling out their current ORAM + TEE solution, they also explored two other approaches:

  1. Using state-of-the-art *classical* private information retrieval (PIR): this approach requires a linear scan wrapped inside cryptography for every query. Signal estimated the cost of this approach to be more than **300M USD per year**, which is not viable for them;
  2. Linear scan + TEE: this approach requires performing a linear scan inside the TEE per batch of queries. Signal deployed this approach in 2017, but required about **500 servers** to handle their load.

  Finally, they rolled out ORAM + TEE in 2022, allowing them to **reduce the number of servers to 6**, achieving almost 100× savings in their computing bill.

  We have been in continuous conversations with Signal. They are interested in migrating to our implementation. To this end, they shared with us a synthetic dataset matching the detailed dimensions of their actual dataset. To assist them in this migration, we are now benchmarking the performance of our implemention using their work loads.

- *Meta.* Meta has launched their own ORAM implementation available through open source [met]. We have been in continuous conversation with the leader of this effort Kevin Lewi. They informed us that

they plan to launch new privacy-preserving AI services using ORAM soon. We will continue our conversations with Meta, with the goal of eventually convincing them to adopt our implementation as well (which promises about $100\times$ speedup over their implementation).

- *Flashbots.* Flashbots has launched a new platform for decentralizing block building. In their BuilderNet [bui], they already provide TEE support. Users can submit encrypted transactions to block builders who then assemble the next block inside the hardware enclave without seeing the transactions in the clear. However, assembling the next block requires speculatively executing the transactions inside the Ethereum Virtual Machine (EVM) — it helps to view each transaction as a smart contract program. The transaction execution incurs accesses to the blockchain's state, and these accesses are highly sensitive, since they leak information such as 1) the user's identity, 2) the crypto-assets traded and 3) the transaction counterparty — see our joint blog with Flashbots for more details [TGSM25]. Just like Signal's private contact discovery, here, the information leaked by access patterns is again the precise information we want to protect in the first place!

  For this reason, Flashbot's roadmap is to add ORAM support to their private block building infrastructure. Because block building is a highly performance-sensitive application, during Phase I, we have been working with them to create demos of our technology tailored for their application, with the primary goal of validating its the performance. As mentioned in Section 1, we have already demonstrated highly encouraging results — if we apply our technology to securely execute all Uniswap transactions contained with in a block, the slowdown per block is under 1ms. We plan to continue working with Flashbots on incorporating ORAM into their private block building infrastructure.

- *Ethereum, ZCash, and other blockchains.* Major blockchains such as Ethereum and ZCash have it in their agenda to provide oblivious accesses to blockchain data to users [But, Conb]. For example, a new privacy roadmap [But] published by Ethereum's founder Vitalik Buterin states Ethereum's commitment to providing privacy of on-chain payments, and anonymization of onchain activity inside applications (e.g., wallets). Providing private reads to the chain through RPC calls is an important goal in this agenda. We have had extensive interactions with Vitalik Buterin, and we are in the process of getting a grant from Ethereum to deliver an ORAM-enabled RPC node for them. ZCash has a very similar roadmap [Conb]: they want users' wallets to fetch shielded transactions from wallet servers, without disclosing which transactions they are interested in. ZCash's promises to protect users' privacy through zero-knowledge proofs at the protocol layer. However, today, this promise is not fully delivered today — as users' wallets fetch their relevant transactions from the wallet server, the access patterns are a giveaway. With these major blockchain players leading the way in providing private accesses to their chain, we expect other blockchain providers to follow the same trend.

- *Private authentication.* 0xPARC and Worldcoin are our existing or soon-to-be partners. Both 0xPARC and Worldcoin are interested in providing privacy-preserving human authentication. For example, Worldcoin, co-founded by OpenAI's Sam Altman, wants to create a universal cryptographic identity called "World ID" that allows users to authenticate to websites and services without disclosing their identities. For example, Worldcoin is partnering with the research arm of the Malaysian government. Maylasia has regulations that require certain websites (e.g., gambling) to perform age verification on their users; however, a seemingly contradictory law prohibits these sites from conducting full Know-Your-Customer checks on their customers. Worldcoin leverages zero-knowledge proofs to enable anonymous authentication, providing a practical solution that allows service providers comply with both legal requirements simultaneously. Unfortunately, Worldcoin's implementation today does not fully deliver their privacy promises. Specifically, as users retrieve their cryptographic credentials from Worldcoin's user database, the access patterns directly reveal the users' identities. To address this issue, Worldcoin is offering us a grant to help them migrate to an oblivious database based on our ENCORE technology.

- *Private database for law firms.* During our Phase I customer discovery, we also interviewed companies building a new IT infrastructure for securely sharing information among law firms. Basically, they want to build a shared database for law firms and their clients to facilitate case and lead discovery (due to our informal confidentiality agreement with them, we will leave the description at this high level). Because such information is of highly sensitive nature, they would like to adopt state-of-the-art cryptographic techniques to secure their information-sharing database and queries to it. Our ENCORE technology, due to its generality and high performance, is a top choice for their application scenario.
- *Decentralized multi-player games.* Our key strategic partner 0xPARC has been partnering with major gaming companies such as EVE Online to build a new generation of decentralized games powered by modern cryptographic techniques such as zero-knowledge proofs (ZKP), and multi-party computation (MPC). As part of this effort, 0xPARC has built demos of decentralized multi-player games that eliminate the need for a trusted centralized server. Through their research and development, they identified oblivious computation as a critical requirement for their use case. Specifically, their multi-player game has a large game state encompassing both a global map and the individual states of all players. Currently, each access to this state requires a linear scan over the entire state wrapped inside cryptography, introducing prohibitive performance overheads. We have been working with them to demonstrate how oblivious computation can help them over this performance bottleneck.
- *Private collaborative AI.* During our customer discovery, we have spoken with several companies that provide collaborative AI solutions to businesses, such as Opaque and Clique. Their customers — spanning sectors such as traditional banking and retail — seek to leverage joint AI analytics by pooling data across multiple organizations or divisions of the same organization. However, they have significant concerns about exposing sensitive business information in the process. Due to its high performance and support for general-purpose computing, our ENCORE technology is ideally positioned to serve as the cornerstone for enabling private collaborative AI.

A.4 Market Size and Growth Trend

There is strong evidence that confidential computing is on track for rapid growth in the coming years, fueled by the rise of privacy-preserving AI in the cloud. Industry leaders are already racing to announce their next-generation privacy-preserving cloud platforms for AI, including Apple's Private Cloud Compute [app], Azure's AI Confidential Inferencing [azua], Google's Confidential Federated Analytics [OR], and Anthropic's Confidential Inference Systems [Ant25]. At the same time, major blockchain ecosystems, such as Ethereum, have committed to integrate privacy into their infrastructure layer [But], which is expected to further accelerate the adoption of privacy-preserving computation in the blockchain space.

Below, we give a market size analysis based on data from various market analyses and forecasts.

- *As an oblivious RPC node for blockchains.* To estimate the market size, we use public statistics for one of Ethereum's major RPC nodes called Infura [infc]. In 2023, Infura's co-founder reported 400,000 registered developers [infb]. Suppose that 10% of these registered developers are active, and pay at the lower tier of $50/month [infd], then Infura's annual revenue is estimated at $24 million per month.

  With ENCORE, we can essentially offer an oblivious counterpart of Infura. According to the performance benchmarking numbers described in Section 1 of our "Project Description", the added latency overhead from obliviousness is negligible relative to the ping latency of the wide-area network (WAN). By offering competitive pricing as the existing non-private services (e.g., Infura, Alchemy), we can convert a significant fraction of the developers and enterprises to our oblivious RPC service instead. Moreover, according to public statistics [infa], Infura's customer base is rapidly growing, e.g., from 100K to 350K in 2021. Therefore, it is reasonable to conclude that just providing an "oblivious RPC node for blockchains" service alone has a market size of at least tens of millions USD per year.

- *As a high-assurance confidential computing technology.* The global confidential computing market size was USD 5.11 billion in 2023. The market is projected to grow USD 417.18 billion by 2032 [RL]. Our company current focuses on the North America market, and the confidential computing market is valued at USD 3.52 billion for the North America market in 2023 [ccc].
- *As a scalable alternative to cryptographic solutions.* The global Multi-Party Computation (MPC) market size is projected to grow from USD 824 million in 2024 to USD 1.4 billion by 2029 [mpc].
- *As an upgrade to today's encrypted databases.* ENCORE-backed databases not only encrypt the data contents but additionally protect the access patterns. Market research suggests that the database encryption market was valued at USD 1.88 billion in 2023, and is projected to grow to USD 11.584 billion by 2032 [dbea]. Today, the database encryption market is dominated by North America, with the government segment holding the largest market share in 2020 at 22.2% share [dbeb].
- *As a high-assurance containerization technology.* The global container technology market size was valued at USD 496.4 million in 2021 and is expected to reach USD 3.1 billion by 2031 [cona].

Market size estimation for oblivious computation. Oblivious computation should not be viewed as merely an upgrade to today's confidential computing and encrypted databases. Instead, it addresses a distinct and indispensable privacy need in many real-world scenarios. In particular, for high-value applications such as securing blockchain or LLM queries, the dataset is often public or already visible to the service provider — what requires protection are the access patterns, not the data itself. This fundamental distinction sets oblivious computation apart from conventional confidential computing technologies and positions it to establish a strong foothold in the market.

Based on the above market research, if oblivious computation emerges as a new industry standard and captures approximately 10% of the confidential computing and encrypted database markets, we project a global market size of $300 million to $1 billion by 2032.

A.5 Barriers to Enter

On one hand, the theory of oblivious algorithms is quite mature from an academic standpoint — for a large class of computations, the best known upper bounds are only a small constant factor away from known lower bounds. On the other hand, there are nonetheless significant barriers towards real-world deployment. Currently the barrier comes from the following factors:

(1) lack of production-ready open-source implementations;

(2) lack of awareness of the broad suite of lower-level oblivious building blocks;

(3) lack of awareness of algorithmic techniques for making oblivious algorithms friendly to real-world architectures (e.g., TEE-based architectures); and

(4) the initial learning curve required to navigate this body of the literature related to (2) and (3);

(5) the error-prone nature of creating secure implementations;

Partly due to these reasons, when Signal started offering private contact discovery, they initially took a detour by implementing a naïve linear scan algorithm, before eventually adopting ORAM [sigb]. Similarly, techniques from the external-memory algorithms literature for enabling enclave-friendly implementations were not widely known among practitioners until our own recent efforts [TGS23, Labb].

Ensuring the security of oblivious implementations in practice remains highly challenging. For instance, Signal's implementation introduced performance optimizations that inadvertently led to information leakage. Specifically, it reveals the number of elements in the stash at any given time. Additionally, in their

sharded version, keys are mapped to shards using a hash function, but the system leaks the number of requests directed to the largest shard [pri]. Both types of leakage depend on the pattern of incoming requests and should be hidden in a secure implementation. Similarly, a well-known academic implementation Oblix [MPC+18] also exhibits leakage due to insufficient care in handling secret-dependent control flow — see our Enigmap paper [TGS23] for a detailed exposition of this issue.

Therefore, although we plan to open source the basic versions of our ORAM and oblivious STL implementations, our company retains significant competitive advantages. Our team includes some of the world's leading experts whose foundational work has played a key role in transitioning oblivious computation toward large-scale adoption. These advantages, which also constitute substantial barriers to entry for potential competitors, stem from our deep expertise and experience in the following areas:

- Algorithmic techniques for adapting ORAM to real-world architectures such as hardware enclaves [TGS23, RS21, fle25, LSX19, CCLS20];
- The rich suite of lower-level oblivious primitives [NWI+15, LWN+15, WNL+14, RS21, LSX19, ACN+20];
- Applying parallel algorithm techniques to oblivious computation [RS21, NWI+15, LWN+15]; and
- The intersection of formal methods and oblivious computation [LHS13, LHH+15, LWN+15] — specifically, in developing optimizing compilers for oblivious computation and leveraging formal methods to programmatically verify the security of oblivious implementations.

Our unique technical strength and capabilities make our team well-positioned to deliver high-performance oblivious algorithms implementations. In particular, we can deliver implementations that enjoy much *better performance and security, with significantly lower R&D costs than other teams.* For example, our oblivious key-value store outperforms those of Signal and Meta by $8\times$ to $106\times$. Further, our implementation enjoys stronger security than Signal's implementation. For businesses and cloud service providers that want to adopt oblivious computation, our team can offer customized high-performance implementations as well as continual engineering support and maintenance at lower costs than potential competitors.

Our team is also widely recognized as the leader in the field of oblivious computation. During our Phase I customer discovery, we also observed that various businesses are interested in working with us not only because of our strong technical capabilities, but also because our long-standing reputation helps them demonstrate the high assurance of the technology to their own customers. For the same reason, we believe that our company is well-positioned to take the lead in establishing an open source consortium for oblivious computation, and attracting donations for the long-term maintenance of the open source libraries, which will serve as a public good for the broader community.

A.6 Technology Objectives and Critical Milestones

The most critical milestone was already accomplished in Phase I, where we established the high throughput and low latency of our technology. This allows us to directly compete with existing non-private services which have established markets, by additionally delivering privacy protections while offering competitive performance and pricing — see also Section C.2.

B Company and Team

Oblivious Labs Inc. was founded in December 2023. We have not taken any venture capital investment. The company is led by our Chief Scientist and President Elaine Shi, and is majority owned by US citizens.

Our mission is to facilitate the wide adoption of oblivious computation, which is exactly what we describe in this proposal. Our vision in the next five years is to make oblivious computation accessible to even non-expert programmers, and establish oblivious computation as an industry best practice.

B.1 Structural and Financial Information

Corporate structure. We are a Delaware C Corp operating in Pittsburgh, Pennsylvania. Our current corporate structure is as follows:

- **Elaine Shi**: Co-founder, president, chief scientist, board director.
- **Afonso Tinoco**: Co-founder, lead research engineer.
- **Tianyao Gu**: Co-founder, worked as lead research engineer in 2024. Currently on leave from the company and full-time enrolled in CMU's Ph.D. program. Will rejoin the company part time in 2026 once he is eligible for CPT again.
- **Andrew Miller**: Scientific advisor.
- **Ping-Lun Wang**: Summer intern.

Our team collaborates with each other closely. The key technical decisions have been jointly been made by Tinoco, Gu, and Shi. Shi currently leads the business development taking into account critical input from Tinoco and Gu. Our scientific advisor Miller not only advises us on our R&D roadmap, but also serves as a key liason between our company and potential customers in the blockchain industry.

Current capitalization. Below is our current cap table. Both Elaine Shi and Andrew Miller are US citizens. Therefore, our company is majority owned by US persons. We have not needed to raise venture capital funding yet.

Stock holder	Common stock	Options	Percent (fully-diluted)
Elaine Shi	1,873,469		58.12%
Afonso Tinoco	600,000		18.61%
Tianyao Gu	150,000		4.65%
Andrew Miller	60,000		1.86%
Options available		540,000	16.75%

Current employee count. Four, including our scientific advisor Andrew Miller.

Financing and revenue history. We have not attempted to raise venture capital investment yet. We have received the following grants and contracts in the past:

- Flashbots (2024-2025): $100,000
- 0xPARC (2024): $40,000 $\times$ 2 = $80,000.
- NSF SBIR Phase I (2024-2025): $274,988.

Sources of operating capital or revenue. At the moment, our operating capital and revenue mainly come from contracts and grants from our key industry partners/customers, as well as federal grants and contracts. We expect other sources of revenue (e.g., subscription to our "oblivious RPC services to public databases" and our PODS containers) to significantly rise in proportion after Phase II — see Section D for more details.

B.2 Key Personnel

PI: Afonso Tinoco. Afonso Tinoco[2] is the PI of the proposed project. He is a Ph.D. student at CMU, and his research focuses on oblivious computation. Tinoco will also oversee the R&D efforts associated with the proposed project, and lead the architectural and algorithmic design.

[2] Afonso Tinoco also goes by the name of "Afonso Tinoco De Faria Cecilio".

Before pursuing his Ph.D., Tinoco worked as a Software Engineer at SingleStore, contributing significantly to the storage and replication layers of a distributed database system. Tinoco obtained his bachelor's and master's degrees at the University of Lisbon – Instituto Superior Tecnico. He was the captain and co-founder of Lisbon cybersecurity CTF team (2015-2023). Tinoco has demonstrated exceptional problem-solving skills and algorithmic proficiency through his participation in both CTF and competitive programming international finals, including CSAW CTF (2018,2019,2021,2023 (2nd place)), European CyberSecurity Challenge (2021), Hack In The Box Pro CTF (2021), RuCTF (2018,2019), VolgaCTF (2017), ACM ICPC Regional SWERC (2015, 2016), and IOI (2012, 2013, 2014).

Tinoco will be working 100% time at Oblivious Labs Inc. at the time of the award. He will spend 95% of his efforts on the proposed project.

Key personnel: Elaine Shi. Shi is a Professor at CMU, an ACM fellow and an IACR fellow. During Phase I, she took one semester of leave from CMU partly to help with Oblivious Labs' technical and business developments. During part of Phase II, she also plans to take either a leave of absence or a sabbatical (most likely 6 - 12 months) from CMU to continue these efforts. Shi has won numerous awards for her work on oblivious computation, including a Packard Fellowship, a Sloan Fellowship, an ONR YIP award, the ACM CCS Test-of-Time award, the NSA Best Scientific Cybersecurity Paper, Top Picks in Hardware and Embedded Security, ASPLOS best paper award, and others.

Key personnel: Tianyao Gu. Tianyao Gu is a Ph.D. student at CMU, conducting research on oblivious computation and applied cryptography. Gu has been a main contributor to our open source libraries [Labb, ost] and several publications on this topic [fle25, GTRS25]. Gu will be working part-time at our company during the performance period, and act as a technical co-lead for the proposed effort. Gu completed his undergraduate studies at University of Michigan and Shanghai Jiao Tong University (SJTU) and his master degree at CMU. He received multiple fellowships including Dean's Fellow at CMU, Roger King Scholarship at University of Michigan, and Undergraduate Excellent Scholarship at SJTU.

No-cost personnel: Andrew Miller (scientific advisor). Andrew Miller is an Adjunct Associate Professor at UIUC. He is also an Associate Director of the Initiative for Cryptocurrencies and Contracts (IC3) and a board member of the Zcash Foundation. He serves as a technical advisor for various cryptocurrency projects. Miller received his Ph.D. from the University of Maryland. He will technically advise our research and engineering team, and serve as a liaison between our company and blockchain companies that need our technology. In particular, Miller has been an influential spokesperson of oblivious computation techniques in the blockchain community.

Besides Miller, we will not have consultants or subawardees for the proposed project.

B.3 Team Qualifications

Expertise on oblivious computation. Our team consists of the world's leading experts on oblivious computation. Our Chief Scientist Shi is a co-author of the Path ORAM [SvDS+13] and Circuit ORAM [WCS15] algorithms currently implemented or deployed by Signal/Meta. Both Path ORAM and Circuit ORAM are instantiations of binary-tree-based ORAMs, an algorithmic paradigm originally invented by Shi [SCSL11].

Our lead research engineers Tinoco and Gu both have extensive experiences in optimizing oblivious computation for both hardware enclaves [TGS23, fle25, Labb, ost] and multi-party computation [GTRS25]. The co-led the development of the fastest key-value store implementation [Labb] known to date, which achieves $8\times$ and $106\times$ speedups over Signal's [sigb] and Meta's implementations [met], respectively. The same team has conducted earlier research at CMU where we maintain the fastest Garbled RAM implementation [GTRS25] known to date, which replaces the TEE with 2-party computation.

Experiences on technology transfer and commercialization. Our Chief Scientist Shi has been involved in

various industry efforts that attempted to transition theory to practice. She consulted for Google as they were rolling out the Rappor privacy-preserving data aggregation system [EPK14] in their Google Chrome. Her work on differentially private prefix sum [CSS10, CSS11] has also been improved and adopted by Google in their privacy-preserving federated learning systems including the Spanish GBoard [ZRX+23].

Shi has been involved in several R&D companies that specialize in transitioning research into practical deployment. Earlier, she worked at Xerox PARC, where she interacted with various industry clients to commercialize AI-based human authentication techniques [SNJC10]. More recently, she has been a visiting professor at 0xPARC to help roll out advanced cryptographic techniques such as zero-knowledge proofs in real-world applications. Shi has also worked as a consultant for Technicolor Research, NTT Research, as well as Andreessen Horowitz's crypto research group.

Shi has been a key influencer in the blockchain community. She co-founded the Initiative for Cryptocurrency and Contracts (IC3) when she was on the tenure-track at Cornell. Earlier, Shi also served as the Chief Scientist in a blockchain startup that aimed to develop scalable, large-scale consensus, and as a scientific advisor to other blockchain companies such as Veridise.

Previously, Tinoco worked at a startup called SingleStore, which developed a high-performance, distributed relational SQL database. In this way, Tinoco gained first-hand experience of bringing cutting-edge storage technology to market. At Oblivious Labs, Tinoco is once again focused on delivering a fast storage solution — this time with built-in privacy protections. His prior experience in launching advanced storage technologies is highly relevant and valuable to this effort.

Last but not the least, our scientific advisor Andrew Miller also has extensive experience in technology transfer and commercialization. He has served on the board of, or as a scientific advisor to numerous blockchain projects. He is a prominent advocate for TEE and oblivious computation in the blockchain community. Miller has been highly effective as a liason, connecting our company with various prospective customers in the blockchain space.

C Product/Technology and Competition

C.1 Features for Enabling a Compelling Value Proposition

As validated in our Phase I efforts, our ENCORE technology simultaneously enjoys strong assurance, high performance, and universality, which jointly contribute to a compelling value proposition. To the best of our knowledge, there is no alternative technology that simultaneously enjoys all these features. In this sense, our technology is necessary and irreplaceable for many of the use cases described in Section A.3.

High performance. Several aspects of performance matter critically to our customers and partners.

- *Low latency.* In many user-facing applications (e.g., contact discovery, digital wallets, authentication), latency must be imperceptible to users. Some use cases, like private order flow, demand even stricter latency guarantees. During our customer discovery interviews with block builders, they emphasized that even millisecond-level delays can impact their profitability. Our TEE-based oblivious key-value store supports each query with only 14 to 16 μs delay on datasets at the scale of Signal and Ethereum — performance unlikely to be matched by purely cryptographic approaches such as multi-party computation (MPC), private information retrieval (PIR), or fully homomorphic encryption (FHE).
- *High throughput.* Many of our existing and prospective customers — such as Signal, Meta, Ethereum, Flashbots, and Worldcoin — handle high volumes of queries. Therefore, they care about the ability to support high throughput with minimum computing resources. Our oblivious key-value store [Labb] delivers 323K queries per second even on a single machine for Ethereum and Signal-scale datasets. Our algorithms are also naturally suited for sharding [CS17, DFD+21], enabling seamless horizontal scaling by adding more servers.

- *Computing cost.* Another often overlooked dimension is the computing cost — an area where our TEE-based oblivious computation holds a significant advantage over alternative cryptographic solutions. For example, Signal estimated that using a state-of-the-art classical PIR scheme — which involves homomorphically evaluating a linear-sized, database-dependent circuit over an encrypted query — would result in an annual computing bill of 300M USD per year, a cost that they cannot afford [rol]. While parallelization can help reduce latency in such schemes, we cannot parallelize away the computing bill.

Strong assurance. In comparison with traditional encrypted databases, our technology is unique in that we additionally protect the access patterns. Recall that in various use cases of Section A.3, the database is either public or already known the service provider. In such cases, encrypting the data contents alone provides no added value. See also the support letters from our partners (including Signal, Ethereum, Flashbots, and Worldcoin), which explain why access pattern privacy is critical in their respective applications.

Universality. Oblivious computation is also valued for its universality, i.e., its ability to protect any general-purpose computation that access large datasets. This universality brings compelling benefits, including *full legacy compatibility* and *ease of software updates*, thus ensuring a low-friction adoption path.

C.2 Customer Willingness to Pay

For our "oblivious access to public databases", we expect the customers to be willing to pay at least as much as today's non-private services [infc, alca]. For example, a low-tier developer subscription costs \$50/month and supports up to 15M queries/day [infd]. Our Phase I results suggest that with a single cloud machine costing about \$1K per month [goob], we can serve 323K queries/sec. At this throughput, a single such machine can service 1860 subscribers. Besides the server rental cost, we estimate the ballpark bandwidth cost to be \$13K under full utilization. Therefore, by offering the same pricing as existing non-private services, we still enjoy a significant profit margin of $\$50 \times \$1860 - \$1\text{K} - \$13\text{K} = \$79\text{K}$ per server. In the blockchain ecosystem, where privacy breaches can result in direct financial risk, we expect that many customers are even willing to pay a premium for enhanced privacy. Nonetheless, our ability to offer competitive pricing can accelerate market conversion and increase our market share.

For our "oblivious computation as a service", our enterprise customers enjoy significantly lower costs subscribing to our services or purchasing our licenses, than hiring in-house experts themselves — partly due to the scarcity of experts in this field (see Section A.5). This has been validated by our existing contracts with industry parnters. Currently, we charge them the fraction of a full-time research engineer's annual salary for what would have cost them five to ten engineer-years effort, had they chosen to develop the same technology in house — see also Section A.5 for the barriers of entry.

C.3 Competitive Landscape

We now compare our ENCORE technology with other confidential computing technologies.

Cryptography-based solutions. Various emerging companies (e.g., Duality technologies, Zama, Inpher) offer cryptography-based approaches to confidential computing, such as Multiparty computation (MPC), Private Information Retrieval (PIR), or Fully Homomorphic Encryption (FHE). Most of today's MPC and FHE offerings rely on the circuit model of computation. Consequently, each memory access triggers cryptographic evaluations over a Boolean or algebraic circuit that encodes the entire dataset, resulting in linear overhead with respect to the database size. For this reason, known production deployments of MPC, FHE, or classical PIR are restricted to small data scenarios. For large datasets, e.g., at Signal's scale and query volume, this linear overhead can lead to computing costs as high as hundreds of millions of dollars per year [rol]. By contrast, our TEE-based oblivious computation delivers near-native performance, and is efficient enough to support even performance-sensitive applications such as block building.

As mentioned in our technical proposal, besides TEE-based oblivious computation, we also plan to develop cryptography-based oblivious computation in the medium to longer-term, thus removing the trust in the TEE. Unlike circuit-model MPC which converts each memory access to a linear-sized circuit, we will rely on oblivious algorithms to support each memory access with only (poly-)logarithmic overhead. Therefore, oblivious computation is also key to scaling MPC to big data.

Encryption at rest. Various cloud services provide encryption at rest, that is, either disk encryption or file-level encryption, e.g., Amazon RDS, Amazon Aurora, Google's Cloud SQL and Cloud Spanner, Azure SQL Database and Cosmos DB, and IBM Cloud Databases. Some cloud services additionally provide encryption in memory by leveraging hardware enclaves, such as AWS Nitro Enclaves, GCP Confidential Computing, IBM Cloud Hyper Protect DBaaS, and Azure SQL Database's "Always Encrypted with secure enclaves" offerings. All these offerings do not protect accesses to data, thus leaking the users' queries and intentions.

Heuristic query sanitization and obfuscation. Several businesses offer heuristic query sanitization and obfuscation. For example, the 1RPC service of Automata [1rp] hides user IP addresses and the type of user agents. Further, 1RPC also mixes multiple users' queries to achieve a k-anonymity style guarantee. The Opaque gateway service uses hardware enclave to sanitize sensitive words from user queries submitted to Large Language Models (LLM). Heuristic approaches may suffer from statistical inference attacks. Further, they may not even be as performant as our provably secure approach. For example, a k-anonymity approach that requires batching multiple users' requests suffers from slow response when requests are sparse.

Differentially private (DP) databases. Several DP database offerings exist, including the Google Differential Privacy Library, IBM Differential Privacy Library, Aircloak Insights, and OpenDP. For using these DP databases, one must trust the platform provider which hosts the data and computation, and the privacy guarantee is only on the released statistics. Our ENCORE technology is complementary to DP databases. Combining our ENCORE framework and a DP database, a business or organization can outsource its data and analytics to an untrusted platform. Not only will the released statistics be private, but also the platform provider cannot break the privacy of each individual.

C.4 Intellectual Property

We will open source our single-machine implementations of ORAM and Oblivious STL. We may choose to retain as intellectual property (IP) our sharded and distributed implementations required for horizontal scaling, and file patents to protect our IP. We do not rely on any external IP.

D Finance and Revenue Model

D.1 Revenue Model

We anticipate the following revenue streams.

1. *Subscription or pay-per-query to oblivious data access services.* We plan to offer oblivious accesses to public databases as a service. For example, for the blockchain space, we can launch a privacy-preserving analog of today's RPC nodes such as Alchemy [alca], Infura [infc], and others [qui], offering APIs for developers to query the blockchain and perform data analytics. Our pricing model will be similar to the existing services [alca, infc, qui], including 1) a free service for end users, 2) a lower-tier developer subscription that supports 4K queries/second and 15M queries/day; as well as 3) higher-tier team and enterprise plans offering much higher throughputs with potentially service-level agreements — see Infura and Alchemy's pricing [infd, alcb] as concrete examples.

2. *Subscription to privacy-enhanced containers through cloud marketplaces.* We plan to make our PODS containers available on major cloud providers through AWS Marketplace, Azure Marketplace, or other similar platforms. This way, businesses can easily spin up a PODS container and add privacy to their their existing services. We plan to adopt a tiered pricing option that is compatible with the pricing models supported by these marketplaces. Specifically, we plan to provide a pay-as-you-go option, and an upfront contract pricing option.
3. *Oblivious computation as a service.* As mentioned in Section A.1, we will offer various forms of support and consultation to our customers including 1) assistance for migrating to our ENCORE technology; 2) customized optimizations tailored to our customers' computation needs; 3) continued engineering support and consultation for software maintenance, updates, and horizontal scaling; 4) workload-specific performance profiling and optimizations; and 5) security review and audits.

   We will make revenue through the following types of mechanisms: 1) *upfront contracts* for one-time services such as developing customized oblivious algorithms, migrating legacy databases to ENCORE, workload-specific performance profiling and optimizations, as well as security review and audits; 2) *subscription fee for recurring services* such as continued engineering support and maintanence; and 3) a *profit-sharing* model, where we charge the customer a percentage of their profit made from the privacy-preserving service for which we provide engineering support and software maintenance.
4. *Licensing.* While the basic, single-machine versions of our oblivious databases and oblivious STL implementations will be publicly released through open source, we plan to develop high-performance, distributed implementations for commercial licensing. Specifically, businesses with large volumes of data will benefit from these distributed versions for the horizontal scaling and enhanced performance.
5. *Donations.* We will keep the single-machine versions of oblivious databases and oblivious STL libraries open source, and create an industry consortium to jointly maintain these open source libraries. Our open-source efforts provide a public good that many companies and businesses can benefit from, and thus accelerate the wider deployment of oblivious computation. Therefore, we expect to receive donations from industry stake-holders as well as individuals for maintaining our open source efforts.

D.2 Staged Finance Plan

Past and current revenue. Since our incorporation in December 2023, we have received grants and contracts from Flashbots ($100,000), 0xPARC ($40,000 $\times$ 2 = $80,000) and NSF SBIR Phase I ($274,988).

Incoming contracts and grants. We expect to soon receive $38,783.60 from Ethereum, roughly $50,000 from Worldcoin, and $80,000 from LayerZero. We are finalizing the paperwork with these companies. We expect these contracts to be renewable on a half-year or yearly basis.

Our existing partners Flashbots and 0xPARC have also expressed interest in continuing a long-term partnership with us, so we expect to receive renewal of contracts from them, estimated at $100,000/yr from Flashbots and $80,000/yr from 0xPARC.

Phase II market expansion plan. During Phase II, we will focus on the following to expand our customer base and generate revenue:

1. *Establish successful use cases with key players.* One priority of Phase II is to help our strategic partners (e.g., Signal, Flashbots, Ethereum, Worldcoin) onboard and launch our technology. With buy-in from these industry leaders, it will help establish oblivious computation as a new industry norm, thus paving the way for our market expansion.
2. *Launch oblivious RPC service for blockchains.* A key priority in Phase II is to launch an oblivious RPC service for blockchains. This critical infrastructure-level service already has an established market

Table 3: Annual pro formas for the next five years.

Category	Year 1	Year 2	Year 3	Year 4	Year 5
Total revenue	\$1M	\$1.85M	\$4M	\$7.5M	\$16M
Grants, contracts, engineering support	\$1M	\$1.5M	\$2M	\$2.5M	\$3M
Oblivious access to public DB	0	\$250K	\$1M	\$3M	\$9M
PODS container (subscription)	0	0	\$250K	\$500K	\$1M
Licensing	0	0	\$250K	\$500K	\$1M
Profit sharing	0	0	\$250K	\$500M	\$1M
Donation	0	\$100K	\$250K	\$500K	\$1M
Cost of Goods Sold (Computing Services)	\$5K	\$37K	\$150K	\$450K	\$1.35M
Total Operating Expenses	\$680K	\$1.37M	\$3.17M	\$4.76M	\$9.12M
Internal R&D	\$400K	\$800K	\$2M	\$3M	\$5M
Sales & Marketing	\$120K	\$240K	\$400K	\$600K	\$900K
General & Administrative	\$156K	\$312K	\$720K	\$1.08M	\$1.77M
Legal	\$10K	\$20K	\$50K	\$80K	\$100K
Earning (EBITDA)	\$309K	\$441K	\$680K	\$2.29M	\$5.53M

although today's services do not offer privacy protection. By delivering robust privacy protections alongside competitive performance and pricing, we can rapidly gain a customer base and generate revenue. After launching this service, we expect a revenue ramp at the end of Year 2 — taking into account the additional time needed for marketing. We anticipate the annual revenue from this service to reach \$1M by the end of Year 3, with a rapid growth trend (see Table 3 and the market size analysis in Section A.4).

3. *Launch* PODS *oblivious container.* We plan to launch this through the marketplace of Azure or Google Cloud by the end of Year 2, and we anticipate a revenue ramp by the end of Year 3 — see Table 3.
4. *Expand "oblivious computation as a service" customer base.* Our past, current, and incoming contracts all fall within the "oblivious computation as a service" category of revenue model. By the end of Phase II, we plan to expand this customer base to $\sim$20 customers, and reach $\sim$\$2M/yr revenue from "oblivious computation as a service" (see our projected Year 3 revenue in Table 3).

Annual pro formas for the next five years. In Table 3, we show our projected revenue and expenses over the next 5 years. The Year 1 and 2 revenue includes the NSF SBIR Phase II budget.

NSF SBIR Phase II funds as an important catalyst. The NSF SBIR Phase II funds will serve as a key enabler for the research and development of our core products and services. During the first two years, we will rely on the NSF funding to break even; but after Year II, we anticipate that our revenue streams from industry contracts and subscriptions should be sufficient for achieving positive cash flow. Specifically, as mentioned, we expect to see significant revenue ramps after the launch of our oblivious RPC service and PODS container, at the end of Years 2 and 3, respectively.

References. See the citations document for our Project Description.